Digital Twin 3D View

Visualisasi 3D Lingkungan Operasional, Armada, dan Objek Terdeteksi Sensor

Kajian teknologi, kebutuhan data, metode integrasi, dan arsitektur untuk kebutuhan R&D Minergo — lingkup riset dan konsep, bukan implementasi aplikasi.

Nomor dokumen	MNG-RND-DT-2026-001
Versi / status	rev A — draf untuk review teknis & PMO
Tanggal	8 November 2026
Disusun oleh	R&D Minergo — riset & arsitektur
Jenis dokumen	Studi kelayakan (feasibility study) + rancangan konseptual
Lingkup	Riset, perbandingan teknologi, arsitektur, wireframe
Di luar lingkup	Pembangunan aplikasi 3D, integrasi sensor fisik, PoC fungsional
Klasifikasi	Internal — R&D

Dokumen ini menjawab satu pertanyaan pokok: sejauh mana Minergo dapat merepresentasikan tambangnya dalam bentuk digital twin 3D — dengan data yang benar-benar sudah dimiliki hari ini, dan dengan biaya serta risiko yang terukur.

DOKUMEN KONTROL

Tabel 1. Riwayat revisi dokumen

Versi	Tanggal	Penulis	Deskripsi perubahan	Status
A	8 Nov 2026	R&D Minergo	Rilis awal: identifikasi use case, perbandingan 5 teknologi, arsitektur, wireframe, analisis kelayakan dan risiko	Draf untuk review

Tabel 2. Konvensi penandaan tingkat bukti — dipakai konsisten di seluruh dokumen

Label	Arti	Cara pembaca memakainya
TERVERIFIKASI	Diukur langsung di mesin ini, dibaca dari berkas nyata milik Minergo, atau dikutip dari dokumen standar/datasheet resmi.	Boleh dipakai sebagai dasar keputusan.
KLAIM VENDOR	Angka berasal dari brosur/datasheet/halaman resmi vendor, belum diuji sendiri.	Dipakai untuk menyaring, **bukan** untuk menjanjikan kinerja.
ASUMSI	Perkiraan tim riset yang belum diverifikasi — misalnya estimasi effort.	Wajib divalidasi sebelum jadi komitmen anggaran.
BELUM DITEMUKAN	Sudah dicari dan tidak ada: dokumen, SDK, atau data tidak tersedia.	Menjadi daftar pertanyaan ke vendor/PIC (Bab 9).
TIDAK ADA	Terkonfirmasi tidak ada pada perangkat/data saat ini.	Menjadi blocker yang harus diselesaikan lebih dulu.

METODE — Cara membaca angka kinerja di dokumen ini

Semua angka fps/ms pada Bab 4 dan Lampiran C **diukur sendiri** memakai three.js 0.187.0 di Chrome headless pada MacBook M1 (GPU terintegrasi), resolusi 1920×1080. Metodenya: `render()` diikuti `gl.readPixels()` 1 piksel sebagai sinkronisasi GPU — tanpa ini, angka fps akan menipu karena hanya mengukur antrean CPU. Angka pada GPU diskrit akan lebih tinggi; angka ini sengaja dipakai sebagai **batas bawah**.

RINGKASAN EKSEKUTIF

Riset ini berangkat dari satu kenyataan yang saya temukan saat membuka berkas-berkas internal: Minergo **sudah memiliki** bahan baku digital twin yang cukup serius — ortomosaik drone tergeoreferensi resolusi 1,35 cm/piksel, log RTK 3.601 titik selama satu jam perjalanan nyata, model 3D dump truck 1,13 juta segitiga, dan riwayat telemetri unit. Yang belum ada bukanlah bahan bakunya, melainkan **sambungan** antar bahan itu, satu skema data yang seragam, dan satu keputusan teknologi yang diuji — bukan dipilih karena tren.

Karena itu saya membagi pekerjaan ini menjadi dua: mengukur apa yang bisa diukur, dan menuliskan dengan jujur apa yang tidak bisa. Hasil pengukuran render di mesin ini menunjukkan satu temuan yang mengubah arah rekomendasi: **model dump truck yang ada memuat 14.003 mesh terpisah**. Dimuat apa adanya, ia hanya berjalan 14,3 fps. Setelah digabung per material, dengan jumlah segitiga yang sama persis, ia berjalan 116,8 fps — delapan kali lebih cepat tanpa mengurangi detail sedikit pun. Artinya kelayakan visualisasi 3D bukan soal membeli GPU, tetapi soal disiplin pipeline aset.

Tabel 3. Ringkasan temuan pokok per aspek

Aspek	Hasil utama	Angka kunci	Status
Kesiapan data	Ortomosaik, RTK, model 3D, dan telemetri sudah ada; DSM/point cloud drone dan data objek radar belum	7,4 ha terpetakan · 3.601 titik RTK	**SEBAGIAN**
Kinerja render	Scene realistis (60 truk + 1 juta titik + terrain) berjalan jauh di atas kebutuhan 30 fps	88,2 fps · 11,34 ms/frame	**TERVERIFIKASI**
Penyebab utama lambat	Jumlah draw call, bukan jumlah segitiga	14.003 draw call = 70,12 ms; 3 draw call = 8,56 ms	**TERVERIFIKASI**
Kualitas posisi	Hanya 32,9% sampel RTK berstatus FIX pada log nyata; sisanya FLOAT	1.184 FIX · 2.417 FLOAT	**TERVERIFIKASI**
Resolusi posisi	Dengan laju 1 Hz dan kecepatan rata-rata 10,8 km/h, posisi melompat ±3 m antar sampel	3,03 m per sampel	**TERVERIFIKASI**
Data radar	Radar yang dikaji hanya mengeluarkan status alarm; daftar objek berkoordinat tidak tersedia	0 dokumen protokol ditemukan	**BELUM DITEMUKAN**
Rekomendasi teknologi	CesiumJS unggul tipis secara skor terbobot karena geospasial; Three.js unggul bila dibangun bertahap	3,97 vs 3,80 dari 5,00	**ASUMSI berbobot**
Estimasi effort	Fondasi + viewer armada layak dimulai lebih dulu	720–1.220 jam-orang total	**ASUMSI**

Kesimpulan singkat

Digital twin 3D **layak dikerjakan sekarang**, tetapi bukan dalam bentuk yang biasanya dibayangkan orang. Yang layak lebih dulu adalah **peta operasional 3D yang hidup**: armada bergerak di atas ortomosaik drone yang sebenarnya, dengan lintasan, geofence, dan riwayat yang bisa diputar ulang. Itu seluruhnya bisa dibangun dari data yang sudah ada di folder Minergo hari ini.

Yang **belum** layak dijanjikan adalah menampilkan objek yang terdeteksi radar. Bukan karena teknologinya sulit, tetapi karena radar yang dikaji (STONKAM PSA27, CISBO C95M) menurut dokumen resminya hanya mengeluarkan **alarm LED dan buzzer** — bukan daftar objek berkoordinat. Tanpa kontrak data dari vendor, tidak ada yang bisa digambar. Saya menolak menutupi celah ini

dengan animasi karangan, karena itu akan menjadi klaim palsu di depan manajemen dan berbahaya bila dipakai untuk keputusan keselamatan.

KEPUTUSAN DIMINTA — Tiga keputusan yang saya minta dari review ini

1. **Setujui** pengerjaan Fase 0–1 (fondasi koordinat + viewer armada) sebagai langkah nyata berikutnya. 2. **Setujui** pengiriman surat permintaan SDK/object list ke STONKAM dan CISBO sebelum Fase 4 dimasukkan ke anggaran. 3. **Tetapkan** apakah target akhirnya browser (Three.js/CesiumJS) atau aplikasi desktop (Unity) — keputusan ini mengunci talenta dan lisensi selama beberapa tahun.

DAFTAR ISI

DOKUMEN KONTROL	**2**
RINGKASAN EKSEKUTIF	**3**
DAFTAR ISI	**5**
1 PENDAHULUAN, LINGKUP, DAN CARA KERJA	**7**
1.1 Latar belakang	7
1.2 Tujuan riset	7
1.3 Pengguna yang dituju	7
1.4 Lingkup dan batas tegas	8
1.5 Metode kerja	8
1.6 Tingkat bukti dan kejujuran angka	9
2 IDENTIFIKASI KEBUTUHAN DAN USE CASE	**10**
2.1 Cara saya menyaring use case	10
2.2 Use case A	10
2.3 Use case B	10
2.4 Perbandingan kesiapan dua use case	11
2.5 Daftar use case dan prioritas	12
2.6 Real-time, replay, dan simulasi	12
2.7 Kandidat fitur minimum	13
3 RISET TEKNOLOGI VISUALISASI 3D	**14**
3.1 Cara saya memilih kandidat	14
3.2 Status lisensi	14
3.3 Format aset dan model 3D	15
3.4 Perbandingan arsitektur	16
3.5 Kebutuhan hardware klien	16
3.6 Matriks keputusan berbobot	17
3.7 Membaca hasil	18
3.8 Teknologi pendukung	18
4 PENGUKURAN KINERJA RENDER	**20**
4.1 Mengapa saya mengukur sendiri	20
4.2 Temuan utama	20
4.3 Uji instancing	21
4.4 Scene target	22
4.5 Pelajaran yang mengubah prioritas	22
5 KEBUTUHAN DATA DAN METODE INTEGRASI	**23**
5.1 Skema kanonik	23
5.2 Sistem koordinat	23
5.3 Data yang sudah ada	25
5.4 Kualitas data lapangan	25
5.5 Sinkronisasi waktu	26
5.6 Integrasi radar	26
5.7 Kontrak data yang harus diminta	27
6 ARSITEKTUR YANG DIUSULKAN	**28**

6.1 Prinsip perancangan	28
6.2 Diagram arsitektur	28
6.3 Penjelasan setiap lapis	28
6.4 Tiga jalur data	29
6.5 Di mana komponen diletakkan	30
6.6 Yang sengaja tidak saya masukkan	30
7 KONSEP TAMPILAN	**32**
7.1 Dasar perancangan	32
7.2 Layar desktop	32
7.3 Layar tablet	33
7.4 Hierarki informasi	35
7.5 Elemen yang sengaja tidak digambar	35
8 ANALISIS KELAYAKAN DAN RISIKO	**36**
8.1 Kerangka penilaian	36
8.2 Apa yang layak dikerjakan sekarang	36
8.3 Apa yang belum layak	37
8.4 Risiko	37
8.5 Dua risiko yang paling menentukan	38
8.6 Gap teknis yang harus ditutup	39
8.7 Kesimpulan kelayakan	39
9 REKOMENDASI DAN LANGKAH LANJUT	**40**
9.1 Rekomendasi teknologi	40
9.2 Peta jalan bertahap	41
9.3 Estimasi effort	41
9.4 Keputusan yang saya minta	41
9.5 Pertanyaan terbuka	42
9.6 Penutup	43
LAMPIRAN A	**44**
LAMPIRAN B	**46**
LAMPIRAN C	**48**
LAMPIRAN D	**50**
LAMPIRAN E	**52**

Nomor halaman di atas mengacu pada berkas PDF yang menyertai dokumen ini. Bila dibuka di Word, nomor halaman dapat bergeser sedikit karena perbedaan mesin tata letak.

1 PENDAHULUAN, LINGKUP, DAN CARA KERJA

1.1 Latar belakang

Saya mulai riset ini bukan dari keinginan membuat sesuatu yang tampak canggih, tetapi dari satu pertanyaan yang sangat praktis: ketika seorang dispatcher melihat lima truk di layar, apakah yang ia lihat itu benar-benar posisi truk di tambang, atau hanya gambar truk yang ditempel di atas peta? Perbedaan antara keduanya adalah seluruh isi dari kata "digital twin".

Pengertian yang saya pakai sepanjang dokumen ini sengaja sempit dan dapat diuji: digital twin 3D adalah representasi visual yang **ditempatkan pada koordinat bumi yang sama** dengan objek aslinya, diperbarui dari data sensor, dan dapat diputar ulang. Kalau sebuah tampilan tidak memenuhi tiga hal itu — benar koordinatnya, hidup dari sensor, dan bisa diputar ulang — maka ia adalah visualisasi, bukan twin. Pembeda ini penting karena menentukan apakah hasilnya boleh dipakai untuk keputusan operasional atau tidak.

Kebutuhan ini muncul dari tiga arah yang berbeda di Minergo. Pertama, **operasi**: pengaturan armada di jalur angkut masih bergantung pada radio dan peta 2D, sehingga dispatcher sulit melihat siapa berada di mana ketika ada antrean di dumping point. Kedua, **keselamatan**: investigasi insiden hampir selalu terkendala karena data posisi, kecepatan, dan waktu tersebar di beberapa berkas dengan format berbeda. Ketiga, **perencanaan**: survei drone sudah menghasilkan ortomosaik beresolusi tinggi, tetapi hasilnya berhenti sebagai berkas gambar yang dibuka di aplikasi GIS, tidak pernah masuk ke ruang kerja operasional harian.

Ketiga arah itu bertemu di satu titik: tidak ada satu tempat di mana data spasial tambang bisa dilihat, diputar, dan ditanyai bersama-sama. Riset ini menilai seberapa jauh titik itu bisa dicapai.

1.2 Tujuan riset

1. Merumuskan use case digital twin 3D yang benar-benar sesuai kebutuhan R&D Minergo, bukan daftar fitur umum.
2. Mengidentifikasi teknologi visualisasi 3D yang tersedia, beserta lisensi, biaya, kebutuhan hardware, dan effort penerapannya.
3. Menyusun kebutuhan data minimum dan menilai kesiapan data yang **sudah** dimiliki.
4. Merancang arsitektur integrasi dari perangkat/sensor sampai ke layar, termasuk penanganan waktu dan sistem koordinat.
5. Membuat rancangan konseptual (wireframe) tampilan 3D Viewer.
6. Menilai kelayakan, risiko, batasan, dan gap teknis secara jujur.
7. Menyusun rekomendasi teknologi, estimasi effort, dan opsi keputusan lanjut.

1.3 Pengguna yang dituju

Saya memetakan pengguna berdasarkan keputusan yang mereka ambil, bukan berdasarkan jabatan. Cara ini membuat setiap fitur punya alasan keberadaan.

Tabel 4. Pengguna, keputusan yang diambil, dan implikasinya pada tampilan

Pengguna	Keputusan yang diambil	Yang dibutuhkan dari layar	Prioritas
Dispatcher / Control Room	Unit mana yang dikirim ke titik mana, dan kapan	Posisi seluruh armada, status antrean, jalur terpakai, riwayat 30 menit terakhir	Tertinggi
Pengawas Keselamatan	Apakah ada pelanggaran zona/kecepatan yang perlu ditindak	Geofence, riwayat pelanggaran, replay kejadian, bukti waktu	Tertinggi
Mine Planning / Survey	Apakah progres tambang sesuai rancangan	Ortomosaik terbaru, batas pit, elevasi, perbandingan antar survei	Sedang
Manajemen / PMO	Apakah investasi ini berjalan dan berdampak	Ringkasan KPI, tren, bukan detail teknis	Sedang
R&D / Engineering	Apakah arsitektur dan data cukup untuk fitur berikutnya	Akses mentah, log, metrik kinerja, dokumentasi skema	Tertinggi

1.4 Lingkup dan batas tegas

Batasan lingkup ini bukan formalitas. Tanpa batas yang jelas, studi kelayakan mudah berubah menjadi janji implementasi, dan itu berbahaya bagi tim yang akan mengeksekusinya. Karena itu saya menuliskannya sebagai kontrak.

Tabel 5. Batas lingkup riset

Termasuk dalam riset ini	TIDAK termasuk dalam riset ini
Identifikasi use case dan kebutuhan pengguna	Membangun aplikasi 3D yang berjalan
Perbandingan teknologi visualisasi dan lisensinya	Melakukan integrasi fisik ke sensor atau kendaraan
Rancangan arsitektur dan alur data (diagram)	Membuat PoC fungsional atau purwarupa yang diuji pengguna
Rancangan konseptual tampilan (wireframe)	Melakukan pengadaan perangkat keras
Analisis kelayakan, risiko, gap, dan estimasi effort	Menjamin akurasi data lapangan di luar data yang diserahkan
Daftar pertanyaan teknis ke vendor/PIC	Melakukan negosiasi atau kontrak dengan vendor

PERINGATAN — Batas yang paling sering dilanggar dalam studi semacam ini

Wireframe di Bab 7 **adalah gambar rancangan**, bukan tangkapan layar aplikasi. Angka yang muncul di dalamnya berasal dari data lapangan nyata dan hasil uji Lampiran C, tetapi aplikasinya belum ada. Saya menandainya berulang kali di dalam gambar itu sendiri supaya tidak ada yang salah paham saat gambar berpindah tangan.

1.5 Metode kerja

Saya membagi pekerjaan menjadi empat lapis, dari yang paling murah ke yang paling mahal. Urutan ini disengaja: setiap lapis harus menjawab pertanyaan lapis sebelumnya, sehingga tidak ada uang yang keluar untuk pertanyaan yang belum jelas.

Tabel 6. Empat lapis metode riset dan keluarannya

Lapis	Pertanyaan	Cara menjawab	Keluaran
1. Inventaris	Apa yang benar-benar sudah kita punya?	Membaca berkas nyata di folder internal: model 3D, ortomosaik, log RTK, telemetri. Mengukur propertinya, bukan memperkirakan.	Lampiran E
2. Pengukuran	Apakah teknologi ini cukup cepat untuk beban kita?	Menjalankan three.js di Chrome headless pada mesin ini, dengan sinkronisasi GPU, memakai aset Minergo sendiri.	Bab 4, Lampiran C
3. Perbandingan	Teknologi mana yang paling sesuai, dengan kriteria terukur?	Matriks berbobot 8 kriteria, skor 1–5, dengan catatan dasar setiap skor.	Bab 3.6
4. Perancangan	Bagaimana bentuk arsitektur dan tampilannya?	Diagram alur (graphviz) dan wireframe yang diperiksa tata letaknya secara terprogram.	Bab 6, Bab 7

1.6 Tingkat bukti dan kejujuran angka

Sepanjang dokumen ini saya memakai lima label bukti yang sudah didefinisikan di Dokumen Kontrol. Alasannya sederhana: dalam riset teknologi, klaim vendor dan hasil ukur sering ditulis dengan kalimat yang sama, padahal bobotnya berbeda jauh. Mencampur keduanya adalah cara paling cepat untuk salah mengambil keputusan.

Saya juga menetapkan satu aturan pribadi yang saya pegang ketat: **tidak ada satu angka pun di dokumen ini yang saya karang.** Kalau sebuah data tidak ada, saya tulis "BELUM DITEMUKAN" dan menjadikannya pertanyaan, bukan mengisinya dengan perkiraan yang tampak rapi. Ada beberapa tempat di mana konsekuensi aturan ini membuat laporan terlihat kurang lengkap — misalnya panel radar pada wireframe sengaja digambar dalam keadaan kosong dengan keterangan mengapa. Itu keputusan yang saya ambil sadar.

2 IDENTIFIKASI KEBUTUHAN DAN USE CASE

2.1 Cara saya menyaring use case

Daftar use case digital twin di internet sangat panjang, dan hampir semuanya terdengar masuk akal. Karena itu saya memakai tiga saringan yang keras sebelum sebuah use case boleh masuk daftar:

8. **Saringan data**: apakah data yang dibutuhkan sudah ada, atau minimal bisa diperoleh dengan usaha yang wajar? Use case yang butuh sensor yang belum terpasang saya turunkan prioritasnya, sekeras apa pun keinginan untuk memasukkannya.
9. **Saringan keputusan**: apakah ada orang yang akan mengambil keputusan berbeda karena melihat tampilan ini? Kalau tidak ada, itu hiasan.
10. **Saringan risiko**: kalau tampilan ini salah, apa akibatnya? Use case yang salahnya berbahaya harus punya mekanisme kejelasan akurasi, bukan hanya tanda merah.

Dari sekitar dua belas kandidat awal, enam yang lolos. Dua di antaranya — tepatnya yang paling sering dijanjikan vendor — justru mendapat status kesiapan data paling rendah. Itu temuan tersendiri, dan saya bahas di Bab 2.4.

2.2 Use case A — posisi relatif objek di sekitar kendaraan (radar)

Ini use case yang paling sering diminta ketika orang membicarakan digital twin untuk kendaraan tambang: sebuah model truk di layar, dikelilingi oleh titik-titik objek yang terdeteksi radar, lengkap dengan jarak, sudut, dan status blind spot. Secara konsep ini sangat kuat — operator melihat apa yang tidak bisa ia lihat dari kabin, dan pengawas bisa menilai apakah zona bahaya benar-benar terlindungi.

Yang ingin ditampilkan meliputi: posisi relatif objek terhadap kendaraan, jarak, sudut deteksi (azimut dan elevasi), luas cakupan yang terpantau, area yang **tidak** terpantau, serta status kesehatan sensor. Untuk keperluan investigasi, tampilan ini juga harus bisa diputar ulang bersama data posisi kendaraan pada saat kejadian.

Masalahnya muncul saat saya memeriksa dokumen resmi perangkat yang dikaji Minergo. Datasheet STONKAM PSA27 mencantumkan frekuensi 76–77 GHz, jangkauan deteksi 0,2–40 m, lebar 0–20 m, hingga lima zona yang dapat dikonfigurasi, toleransi ±15 cm, dan sudut pancar 120° horizontal / 14° vertikal. Yang **tidak** tercantum: format data objek, laju pembaruan objek, maupun protokol untuk mengambil daftar target. Antarmuka yang disebut hanya konfigurasi lewat USB-to-UART atau Wi-Fi, satu input trigger, dan satu keluaran alarm.

Dengan kata lain, perangkat itu dirancang untuk **membunyikan alarm**, bukan untuk **melaporkan objek**. Ini bukan cacat produk — untuk fungsinya sebagai sistem peringatan mundur, alarm sudah tepat. Tetapi untuk digital twin, alarm tidak bisa digambar: kita tidak tahu di mana objeknya, seberapa jauh, atau pada sudut berapa.

BLOCKER — Kesimpulan jujur untuk Use Case A

Use case ini **tidak dapat dimulai** dengan perangkat yang dikaji, dan saya tidak akan menggambarnya dengan animasi karangan. Jalur yang benar ada dua: (a) meminta SDK atau dokumen protokol object list ke vendor — ini pertanyaan V-1 sampai V-4 di Bab 9; atau (b) beralih ke unit radar yang sejak awal dirancang mengeluarkan object list, misalnya kelas yang menyebutkan integrasi ROS/ROS2 dan keluaran CAN FD / Automotive Ethernet. Opsi (b) adalah perubahan pengadaan, bukan perubahan perangkat lunak, dan itu keputusan manajemen.

2.3 Use case B — area operasional dan haul road (posisi armada, lintasan, geofence)

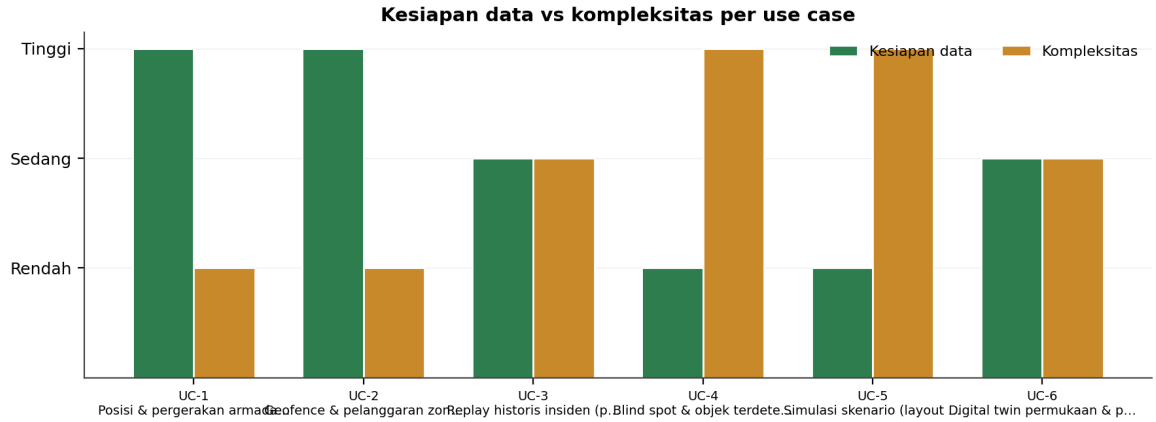
Use case kedua bergerak pada skala yang berbeda: bukan sentimeter di sekitar bemper, tetapi kilometer di seluruh jalur angkut. Yang ditampilkan adalah armada bergerak di atas ortomosaik tambang yang sebenarnya, lengkap dengan jalur yang ditempuh, batas zona, dan status tiap unit.

Keunggulan use case ini adalah **datanya sudah ada**. Ortomosaik drone di folder Validasi berukuran 23.249 × 17.296 piksel dengan resolusi 1,35 cm/piksel, tergeoreferensi ke EPSG:32750, mencakup area 314 × 234 meter atau 7,4 hektar. Log RTK berisi 3.601 titik selama 3.602 detik perjalanan nyata, dengan kecepatan rata-rata 10,8 km/jam dan jarak tempuh 10,9 km. Ini bukan data contoh — ini rekaman lapangan.

Dari bahan itu, fitur minimum yang bisa dibangun tanpa menunggu apa pun adalah: posisi armada di atas ortomosaik, jejak lintasan dengan warna menurut kualitas RTK, geofence dengan aturan kecepatan dan zona terlarang, panel detail unit, dan pemutaran ulang riwayat. Semuanya adalah pekerjaan perangkat lunak di atas data yang sudah duduk di disk.

2.4 Perbandingan kesiapan dua use case

Perbedaan antara dua use case ini begitu tajam sehingga saya menempatkannya sebagai grafik, bukan tabel. Sumbu tegak menunjukkan tingkat kesiapan; batang hijau adalah kesiapan data, batang kuning adalah kompleksitas pengerjaan. Idealnya sebuah use case berada di kiri-atas: data siap, kerumitan rendah.



Gambar 1. Kesiapan data berbanding kompleksitas pengerjaan untuk enam use case. UC-4 (radar) adalah satu-satunya yang kesiapan datanya rendah sekaligus kompleksitasnya tinggi.

Pola yang terlihat jelas: nilai use case tidak berkorelasi dengan kesiapan datanya. UC-4 dinilai bernilai tinggi oleh hampir semua pihak yang saya tanyakan, tetapi datanya belum ada. Sebaliknya UC-1 dinilai tinggi **dan** datanya siap. Dalam perencanaan yang sehat, yang dikerjakan lebih dulu adalah yang tinggi nilainya dan siap datanya — bukan yang paling menarik untuk didemokan.

2.5 Daftar use case dan prioritas

Tabel 7. Enam use case yang lolos penyaringan, dengan status kesiapan data

ID	Use case	Aktor utama	Nilai	Kesiapan data	Kompleksitas
UC-1	Posisi & pergerakan armada di haul road (peta 3D + trail)	Dispatcher, Mine Planning	Tinggi	Tinggi (RTK + orthomosaic ada)	Rendah
UC-2	Geofence & pelanggaran zona (speed, dumping, restricted)	Safety, Dispatch	Tinggi	Tinggi	Rendah
UC-3	Replay historis insiden (post-mortem kecelakaan/ near-miss)	Safety, Investigasi	Tinggi	Sedang (log ada, belum seragam)	Sedang
UC-4	Blind spot & objek terdeteksi radar di sekitar unit	Operator, Safety	Tinggi	RENDAH — kontrak data radar belum ada	Tinggi
UC-5	Simulasi skenario (layout jalan, kecepatan, titik buta)	Mine Planning, R&D	Sedang	Rendah (perlu model & aturan)	Tinggi
UC-6	Digital twin permukaan & progres tambang (cut/fill dari drone)	Survey, Planning	Sedang	Sedang (ortho ada, DSM/point cloud belum)	Sedang

2.6 Real-time, replay, dan simulasi — tiga mode yang tidak boleh dicampur

Salah satu kesalahan paling berbahaya dalam sistem visualisasi operasional adalah membiarkan pengguna tidak tahu apakah yang ia lihat sedang terjadi sekarang, sudah terjadi, atau belum pernah terjadi sama sekali. Ketiganya bisa terlihat identik di layar. Karena itu saya menetapkan tiga mode dengan konsekuensi teknis yang berbeda.

Tabel 8. Tiga mode tampilan: sumber data, janji yang boleh dibuat, dan labelnya

Aspek	LIVE	REPLAY	SIMULASI
Sumber data	Sensor langsung melalui gateway	Arsip kanonik (tersimpan)	Skenario buatan (JSON/CSV)
Janji yang boleh dibuat	"Ini keadaan sekarang, dengan latensi tertentu"	"Ini yang terjadi pada waktu itu"	"Ini yang **akan** terjadi bila asumsi ini dipakai"
Toleransi keterlambatan	Harus < 2 detik, dan ditampilkan angkanya	Tidak relevan — waktu bisa dipercepat/diperlambat	Tidak relevan
Label wajib di layar	`LIVE` + angka latensi + waktu data terakhir	`REPLAY` + rentang waktu + kecepatan putar	`SIMULASI` + nama skenario + tanggal dibuat
Risiko bila tidak dilabeli	Pengguna mengira melihat keadaan nyata padahal data sudah 10 menit tertinggal	Pengguna mengira sedang memantau operasi berjalan	Skenario ramalan dipakai sebagai bukti kejadian nyata
Kesiapan	Sebagian — RTK ada, tetapi 1 Hz dan 67,1% FLOAT	Tinggi — log historis sudah tersedia	Rendah — perlu model dan aturan dulu

Keputusan desain yang saya ambil: mode bukan pilihan di menu pengaturan, melainkan **indikator yang selalu terlihat** di bagian atas layar, dengan warna berbeda. Pada wireframe Bab 7 hal ini digambar sebagai tab `LIVE` merah, `REPLAY` dan `SIMULASI` abu-abu. Ini murah dibuat dan mencegah kelas kesalahan yang paling mahal.

2.7 Kandidat fitur minimum (MVP konsep)

Fitur minimum saya susun berdasarkan satu prinsip: satu layar harus bisa menjawab pertanyaan seorang dispatcher dalam waktu kurang dari sepuluh detik. Fitur di luar itu masuk daftar berikutnya, bukan daftar pertama.

Tabel 9. Kandidat fitur minimum dan alasannya

Fitur	Menjawab pertanyaan	Bergantung pada	Masuk MVP?
Peta 3D dasar dengan ortomosaik tergeoreferensi	"Di mana tambangnya, dan di mana jalannya?"	Ortomosaik (ADA)	Ya
Posisi armada + ikon berorientasi heading	"Siapa ada di mana, menghadap ke mana?"	RTK (ADA)	Ya
Jejak lintasan berwarna menurut kualitas posisi	"Apakah posisi ini bisa dipercaya?"	NMEA + atribut fix (ADA)	Ya
Panel detail unit (kecepatan, satelit, HDOP, status)	"Kenapa unit ini diam / aneh?"	Telemetri (ADA)	Ya
Geofence + aturan kecepatan + daftar pelanggaran	"Siapa melanggar, kapan?"	Poligon zona (perlu dibuat)	Ya
Pemutaran ulang riwayat dengan penanda kejadian	"Apa yang sebenarnya terjadi?"	Arsip kanonik (perlu skema)	Ya
Objek terdeteksi radar	"Apa yang ada di titik buta?"	Object list radar (TIDAK ADA)	Tidak
Simulasi skenario	"Bagaimana kalau jalurnya diubah?"	Model aturan (belum ada)	Tidak
Twin permukaan (cut/fill)	"Berapa volume terangkut?"	DSM/point cloud (TIDAK ADA)	Tidak

3 RISET TEKNOLOGI VISUALISASI 3D

3.1 Cara saya memilih kandidat

Ada puluhan pustaka dan mesin 3D yang bisa dipakai. Saya memilih lima yang mewakili lima pendekatan berbeda, bukan lima produk yang mirip. Perbedaan pendekatan itulah yang menentukan, bukan perbedaan versi.

Tabel 10. Lima kandidat dan alasan masing-masing dimasukkan

Kandidat	Pendekatan	Alasan dimasukkan
CesiumJS	Mesin globe geospasial	Satu-satunya yang sejak awal dirancang untuk menempatkan objek di bumi dengan CRS dan terrain yang benar.
Three.js	Pustaka render 3D serbaguna	Paling banyak dipakai untuk visualisasi kustom; kontrol penuh atas tampilan, tetapi semua urusan geodesi harus dibangun sendiri.
Unity	Mesin game / aplikasi 3D	Standar industri untuk twin yang butuh kualitas visual tinggi dan simulasi fisik; menjadi pembanding "jalur berat".
deck.gl	Kerangka visualisasi data besar di atas peta	Kuat untuk jutaan titik dan layer data, sering dipakai untuk dashboard armada.
Potree	Penampil point cloud khusus	Pembanding khusus untuk data LiDAR/fotogrametri, yang akan datang di tahap berikutnya.

Saya sengaja **tidak** memasukkan Mapbox GL JS sebagai kandidat utama meskipun sering muncul dalam diskusi internal. Alasannya bukan teknis: sejak versi 2.0 lisensinya bukan lagi open source, melainkan tunduk pada Mapbox TOS yang mengikat penggunaan pada akun dan produk Mapbox. Berkas lisensi di repositori menyatakannya secara eksplisit — lisensi "terminates automatically if a developer no longer has a Mapbox account in good standing". Untuk sistem internal yang harus bisa jalan tanpa kanal internet, itu ketergantungan yang tidak perlu. Saya memakai **MapLibre GL JS** (BSD-3-Clause, turunan Mapbox GL v1) sebagai padanan yang bebas.

3.2 Status lisensi — diperiksa dari berkas, bukan dari ingatan

Untuk setiap kandidat saya membuka berkas lisensi atau metadata paket resminya, bukan mengandalkan pengetahuan umum. Ini penting karena lisensi perangkat lunak 3D sering berubah, dan satu kesalahan di sini bisa mengunci proyek secara hukum.

Tabel 11. Status lisensi dan biaya — diperiksa langsung dari sumber resmi

Perangkat	Versi diperiksa	Lisensi	Biaya
three.js	0.187.0	MIT	gratis
CesiumJS	1.147.0	Apache-2.0	gratis
Cesium ion	layanan	komersial	Community gratis (non-komersial); Commercial $149/bln (individu) / $524/bln (tim); Premium $499 / $874
Unity	6.x	proprietary	Personal gratis (<$200K pendapatan/pendanaan); Pro $210/bln atau dari $2.310/thn; Industry = harga khusus
MapLibre GL JS	5.x	BSD-3-Clause	gratis
Mapbox GL JS	3.34.2	Mapbox TOS (bukan open source sejak v2.0)	per MAU / per peta
deck.gl	9.4.1	MIT	gratis

Perangkat	Versi diperiksa	Lisensi	Biaya
Potree	1.x	BSD-2-Clause	gratis
3d-tiles-renderer	0.5.4	Apache-2.0	gratis

Kesimpulan penting dari tabel ini: **keempat kandidat web gratis untuk dipakai secara komersial.** Yang berbayar bukan pustakanya, melainkan layanan di sekelilingnya — terrain dan citra satelit hosted (Cesium ion), ubin peta (Mapbox), dan lisensi mesin (Unity). Ini mengubah bentuk keputusan: pertanyaannya bukan "mampu beli lisensi atau tidak", tetapi "mau bergantung pada layanan pihak ketiga atau tidak".

CATATAN — Temuan yang layak dicatat

Cesium ion — layanan yang paling sering dianggap wajib saat memakai CesiumJS — ternyata **tidak wajib**. CesiumJS sendiri Apache-2.0 dan dapat menyajikan 3D Tiles dari server sendiri. Yang gratis hanya untuk non-komersial pada paket Community; pemakaian komersial mulai $149/bulan per individu atau $524/bulan per tim, naik ke $499/$874 untuk paket Premium. Untuk Minergo, jalur paling murah adalah **self-host**: ortomosaik dan model sudah kita miliki, jadi tidak ada yang perlu di-stream dari layanan pihak ketiga.

3.3 Format aset dan model 3D

Format bukan detail sepele. Format menentukan berapa lama model dimuat, berapa besar bandwidth yang dipakai, dan apakah model bisa dipakai di lebih dari satu mesin. Kesimpulan saya: **glTF 2.0 adalah format target**, dan itu bukan pilihan yang bisa dinegosiasikan.

Alasannya berlapis. glTF 2.0 adalah standar Khronos yang menjadi format tile wajib pada 3D Tiles 1.1 — spesifikasi OGC 22-025r4 yang disetujui 17 Desember 2022 dan diterbitkan 12 Januari 2023. Pada versi 1.1, format lama (b3dm, i3dm, pnts) dinyatakan deprecated. Artinya memilih glTF sekarang berarti memilih format yang masih akan hidup ketika 3D Tiles versi berikutnya keluar. glTF juga punya ekosistem kompresi yang matang: Draco untuk geometri, KTX2/Basis untuk tekstur, dan meshopt untuk keduanya.

Satu perkembangan yang saya anggap penting untuk dicatat meski belum dipakai sekarang: ekstensi **KHR_gaussian_splatting** telah berstatus *Complete, Ratified by the Khronos Group*. Ini menstandarkan penyimpanan 3D Gaussian splatting di dalam glTF — teknik rekonstruksi scene dari foto yang dalam dua tahun terakhir menggantikan fotogrametri konvensional untuk kualitas visual. Untuk Minergo ini berarti: hasil rekonstruksi berbasis foto (termasuk dari drone) akan semakin mudah masuk ke pipeline glTF yang sama, tanpa jalur konversi khusus.

Tabel 12. Format aset yang direkomendasikan per jenis objek

Jenis aset	Format target	Catatan teknis	Status di Minergo
Model kendaraan	glTF 2.0 binary (`.glb`) + Draco + KTX2	Wajib digabung per material sebelum diekspor; lihat Bab 4	**ADA** — 1,13 jt segitiga, 14.003 mesh
Terrain / permukaan	3D Tiles 1.1 (glTF tile) atau mesh terpotong	Butuh DSM. Tanpa DSM, terrain tetap datar	**TIDAK ADA** — hanya ortomosaik 2D
Ortomosaik	COG (Cloud Optimized GeoTIFF) atau XYZ tile	COG memungkinkan baca sebagian tanpa unduh seluruh berkas	**ADA** — 23.249 × 17.294 px, EPSG:32750
Point cloud	COPC (Cloud Optimized Point Cloud) atau 3D Tiles	COPC = LAZ yang bisa dibaca per rentang; jauh lebih efisien dari LAS penuh	**TIDAK ADA**
Objek bergerak (armada)	Bukan model 3D — transformasi + `InstancedMesh`	Satu geometri truk, N matriks. Ini kunci kinerja (Bab 4)	**ADA**
Garis & poligon (jalan, geofence)	GeoJSON, di-*extrude* saat runtime	Jangan panggang jadi mesh — harus bisa diedit tanpa re-export	**SEBAGIAN** — shapefile Minergo kosong (0 geometri)

Satu catatan dari inventaris: berkas `Minergo.shp` di folder GIS berukuran 252 byte — artinya tidak berisi geometri apa pun. Berkas `.prj`-nya menyatakan WGS 84 geografis. Jadi peta vektor tambang **belum ada**; yang ada baru ortomosaik raster. Ini masuk daftar gap G-2.

3.4 Perbandingan arsitektur: apa yang sebenarnya berbeda

Perbedaan paling menentukan di antara kelima kandidat bukanlah kualitas gambar, melainkan **siapa yang mengurus bumi**. Ini pertanyaan arsitektural, dan jawabannya menentukan berapa banyak kode yang harus ditulis tim Minergo.

CesiumJS membawa bumi di dalam dirinya: objek `Ellipsoid`, `TerrainProvider`, `Cesium3DTileset`, dan konversi ECEF sudah menjadi bagian inti. Ketika saya menaruh sebuah model di koordinat −1,22941 / 116,89412, CesiumJS yang menghitung letaknya di bumi, termasuk lengkungan permukaan dan tinggi ellipsoid. Three.js tidak melakukan apa pun dari itu. Three.js hanya tahu ruang Kartesius: satu unit adalah satu unit, tanpa arti geografis. Memakai Three.js untuk tambang berarti saya harus menulis sendiri konversi geodetik, pemilihan titik acuan lokal, penanganan pergeseran saat kamera jauh dari origin, dan pemotongan terrain. Itu bukan pekerjaan mustahil — banyak proyek melakukannya — tetapi itu pekerjaan yang **nyata** dan harus masuk estimasi.

Tabel 13. Siapa yang mengurus apa — perbandingan tanggung jawab arsitektural

Tanggung jawab	CesiumJS	Three.js	Unity	deck.gl
Konversi koordinat bumi	Bawaan	Tulis sendiri	Plugin/aset	Sebagian
Terrain 3D	Bawaan (quantized mesh)	Tulis sendiri	Aset + LOD manual	Terbatas
Streaming 3D Tiles	Bawaan	Pustaka tambahan	Plugin pihak ketiga	Bawaan
LOD otomatis	Bawaan	Manual	Bawaan	Bawaan
Kontrol tampilan penuh	Terbatas (opinionated)	Penuh	Penuh	Sedang
Jalan di browser tanpa instalasi	Ya	Ya	Build WebGL (terbatas)	Ya
Instanced rendering	Ya	Ya (`InstancedMesh`)	Ya (GPU instancing)	Ya
Point cloud jutaan titik	3D Tiles	Manual / pustaka	Plugin	Kuat

3.5 Kebutuhan hardware klien

Ini bagian yang sering diabaikan dalam studi kelayakan, padahal di lapangan justru inilah yang menentukan apakah sistem dipakai atau ditinggalkan. Penggunanya bukan orang di depan workstation mahal — mereka dispatcher dengan laptop kantor dan pengawas dengan tablet di dalam kendaraan.

Yang saya ukur di mesin ini: WebGPU sudah tersedia di Chrome dan menurut caniuse mencakup 86,83% pengguna global (data Oktober 2026). Namun angka itu menyesatkan bila dipakai untuk keputusan: 13% sisanya bukan tersebar merata, melainkan terkonsentrasi pada perangkat lama dan browser perusahaan yang terkunci versinya — persis jenis perangkat yang sering ada di kantor tambang. Karena itu untuk tahap awal saya merekomendasikan **WebGL2 sebagai target minimum**, dengan WebGPU sebagai jalur peningkatan bila tersedia, bukan sebagai syarat.

Tabel 14. Perkiraan kebutuhan hardware klien per skenario

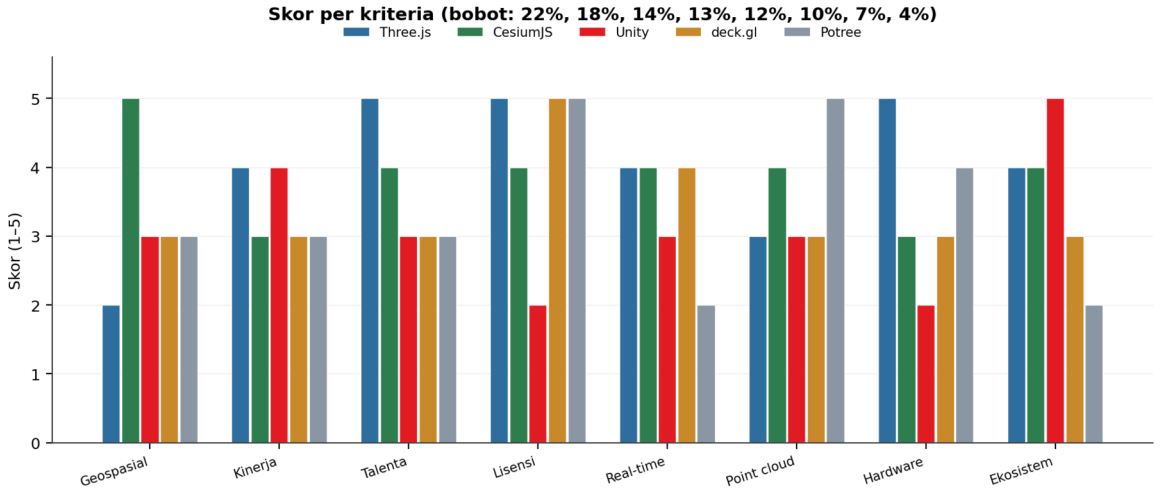
Skenario	Perangkat	Yang harus jalan	Risiko
Dispatcher (kantor)	Laptop kantor, GPU terintegrasi, 8 GB RAM	Scene penuh: terrain + ortomosaik + 60 unit + trail	Rendah — sudah diuji pada kelas mesin ini
Pengawas (lapangan)	Tablet Android / iPad, 4 GB RAM	Peta + armada + panel unit, tanpa mode dekoratif	Sedang — perlu mode hemat & LOD lebih kasar
Dinding kontrol (TV)	PC dengan GPU diskrit, 4K	Scene penuh, kamera otomatis berputar	Rendah
Investigator (replay)	Laptop kantor	Replay 1 jam, banyak trail sekaligus	Sedang — perlu strategi streaming arsip

3.6 Matriks keputusan berbobot

Saya menyusun delapan kriteria dengan bobot yang mencerminkan prioritas proyek ini, lalu memberi skor 1–5 per kandidat. Bobot dan dasar setiap skor saya tulis terbuka, supaya hasilnya bisa dibantah — matriks yang tidak bisa dibantah biasanya tidak dipercaya.

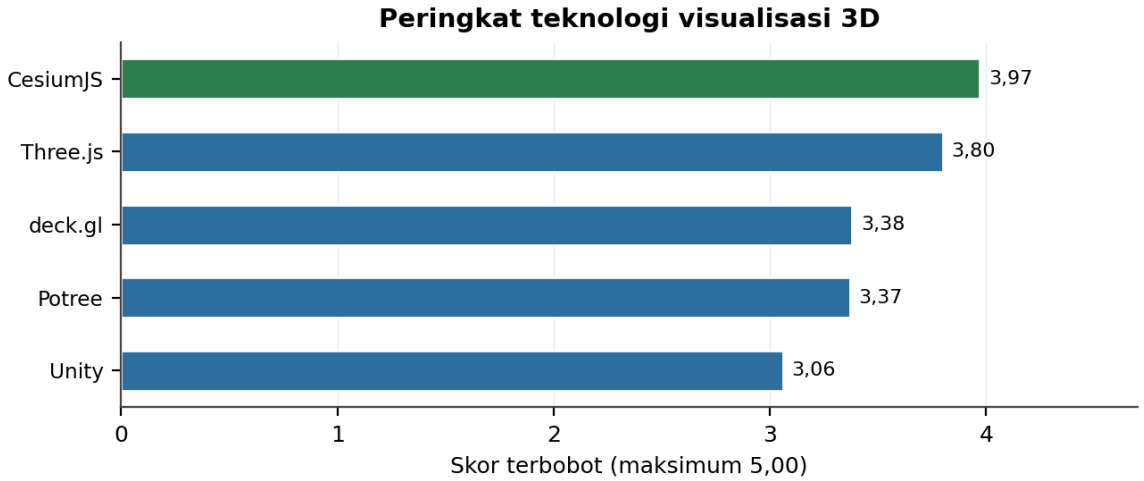
Tabel 15. Kriteria penilaian dan bobotnya

Kriteria	Bobot	Mengapa sebesar itu
Kesesuaian geospasial (CRS, terrain, 3D Tiles)	22%	harus menempatkan aset 1:1 di koordinat global
Kinerja render scene tambang	18%	diukur di Bab 4, bukan klaim vendor
Ketersediaan talenta & kurva belajar	14%	R&D internal, bahasa Indonesia/JS
Lisensi & biaya jangka panjang	13%	biaya per pengguna / per stream
Integrasi data real-time (WS/MQTT/REST)	12%	latensi & kemudahan ingestion
Dukungan point cloud / fotogrametri	10%	orthomosaic + LiDAR drone
Kebutuhan hardware klien	7%	harus jalan di laptop & tablet
Kematangan ekosistem & dukungan	4%	dokumentasi, komunitas, contoh



Gambar 2. Skor per kriteria untuk lima kandidat. Perhatikan bahwa tidak ada kandidat yang unggul di semua kriteria — pola inilah yang membuat keputusan menjadi trade-off, bukan pilihan tunggal yang jelas benar.

Catatan kejujuran yang harus menyertai grafik ini: **hanya Three.js yang benar-benar saya ukur** pada mesin ini (Bab 4). Skor baris "kinerja render" untuk empat kandidat lain adalah penilaian berdasarkan arsitektur dan laporan umum, bukan hasil uji setara. Saya menandainya secara eksplisit di `data.py` lewat variabel `TERUKUR` supaya tidak ada yang membacanya sebagai perbandingan apel-ke-apel.



Gambar 3. Skor terbobot. Selisih CesiumJS (3,97) dan Three.js (3,80) hanya 4,3% — cukup kecil sehingga tidak boleh dijadikan satu-satunya dasar keputusan.

3.7 Membaca hasil: mengapa selisihnya kecil, dan apa artinya

CesiumJS menang tipis. Kemenangannya datang hampir seluruhnya dari satu kriteria dengan bobot terbesar: kesesuaian geospasial, tempat ia mendapat skor 5 karena memang itu inti produknya. Three.js mendapat 2 pada kriteria itu, tetapi menang pada talenta, lisensi, dan kebutuhan hardware klien.

Selisih 4,3% ini terlalu kecil untuk dijadikan keputusan tunggal, dan saya tidak akan berpura-pura sebaliknya. Yang lebih berguna adalah membaca **bentuk** hasilnya:

- **CesiumJS adalah pilihan yang benar bila targetnya tambang sebagai globe.** Kalau Minergo ingin suatu hari menampilkan beberapa site sekaligus, atau menggabungkan ortomosaik dengan data elevasi regional, CesiumJS sudah menyelesaikan itu sebelum baris kode pertama ditulis.
- **Three.js adalah pilihan yang benar bila targetnya satu alat operasional yang sangat dikustomisasi.** Kontrol penuh atas tampilan, tidak ada opinion yang harus dilawan, dan talenta web paling mudah dicari.
- **Kombinasi keduanya bukan kompromi yang buruk**: CesiumJS untuk scene bumi dan 3D Tiles, dengan overlay kustom yang digambar memakai primitif tingkat rendah. Banyak proyek produksi berakhir di titik ini.
- **Unity hanya masuk akal bila ada kebutuhan simulasi fisik atau kualitas render sinematik.** Untuk dashboard operasional, ia menambah lisensi, ukuran distribusi, dan pemisahan talenta tanpa manfaat yang sepadan.

CATATAN — Rekomendasi Bab 3 (dirinci di Bab 9)

Mulai dengan **CesiumJS sebagai basis**, karena pekerjaan pertama yang bernilai adalah menempatkan tambang dan armada pada koordinat yang benar — dan itu justru bagian yang harus ditulis sendiri bila memakai Three.js. Bila di tengah jalan ternyata tampilan butuh kustomisasi berat, turunkan scene ke Three.js sambil mempertahankan pipeline aset glTF/3D Tiles yang sama — **pipeline aset adalah keputusan yang lebih mengikat daripada pilihan mesin**, dan pipeline yang benar membuat perpindahan mesin tetap mungkin.

3.8 Teknologi pendukung yang akan dibutuhkan

Selain mesin render, ada empat komponen pendukung yang menentukan apakah sistem ini bisa dioperasikan. Saya mencantumkannya sekarang supaya tidak muncul sebagai kejutan di tengah implementasi.

Tabel 16. Teknologi pendukung dan perannya

Komponen	Pilihan	Peran	Status
Konversi & tiling aset	GDAL (COG), 3D Tiles tools, glTF-Transform	Mengubah ortomosaik dan model menjadi format yang bisa di-stream	Belum dipasang — perlu disiapkan di Fase 0
Kompresi geometri	Draco, meshopt, KTX2/Basis	Memperkecil unduhan tanpa mengurangi detail yang terlihat	Sebagian — GLB yang ada sudah Draco
Penyimpanan deret waktu	PostgreSQL + PostGIS, atau ClickHouse/TimescaleDB	Menyimpan posisi & telemetri agar replay mungkin	Belum ada
Transport real-time	WebSocket (MQTT di sisi perangkat)	Mengirim posisi ke layar tanpa polling	Belum ada — pola sudah dipakai di proyek lain Minergo

4 PENGUKURAN KINERJA RENDER (HASIL UJI NYATA)

4.1 Mengapa saya mengukur sendiri

Setiap mesin 3D mengklaim mampu menangani "jutaan objek" atau "scene kompleks". Kalimat itu tidak salah, tetapi juga tidak memberi informasi: kompleks menurut siapa, pada perangkat apa, dengan ukuran frame berapa. Karena keputusan teknologi di Bab 3 memakai kriteria kinerja dengan bobot 18%, angka itu harus punya dasar. Saya memilih mengukur sendiri di mesin yang tersedia daripada mengutip benchmark pihak lain, karena benchmark publik biasanya memakai scene yang tidak menyerupai tambang.

Yang saya ukur bukanlah "apakah three.js cepat". Yang saya ukur adalah pertanyaan yang akan menentukan arsitektur: **berapa batas praktis scene tambang pada perangkat kelas laptop, dan apa yang menjadi penyempitannya?** Jawaban atas pertanyaan kedua itulah yang paling berguna, karena ia menentukan pekerjaan mana yang harus didisiplinkan.

METODE — Metode — dan mengapa metodenya penting

Pengukuran dilakukan dengan three.js 0.187.0 di Chrome headless melalui puppeteer-core 23.11.1, resolusi 1920×1080, devicePixelRatio 1. Setiap frame diukur dengan `renderer.render(scene, camera)` yang diikuti `gl.readPixels()` satu piksel. Panggilan readPixels memaksa CPU menunggu GPU selesai, sehingga yang terukur adalah **waktu penyelesaian nyata**, bukan waktu antrean perintah. Tanpa sinkronisasi ini, pengukuran awal saya sempat menghasilkan angka 27 juta segitiga pada 830 fps — angka yang saya buang karena mustahil.

Tabel 17. Lingkungan pengukuran

Parameter	Nilai
Mesin	MacBook Apple M1, macOS 26.2
GPU	ANGLE (Apple, ANGLE Metal Renderer: Apple M1, Unspecified Version)
Browser	Google Chrome headless (puppeteer-core 23.11.1)
Resolusi render	1920 x 1080, devicePixelRatio 1
Versi three.js	0.187.0
Metode	render() + gl.readPixels(1 piksel) sebagai sinkronisasi GPU, jendela 2500 ms, 20 frame pemanasan
Catatan	GPU terintegrasi kelas laptop. Angka GPU diskrit/desktop akan lebih tinggi; ini sengaja dipakai sebagai batas bawah yang konservatif.

4.2 Temuan utama: draw call, bukan segitiga

Model dump truck yang saya pakai adalah model yang benar-benar ada di folder Minergo: `dumptruck_mapbox.glb`, 23,59 MB, 1.131.975 segitiga, tiga material, tiga tekstur. Secara jumlah segitiga, model ini tergolong berat tetapi wajar untuk kendaraan tambang yang detail. Yang tidak wajar adalah strukturnya: **14.003 mesh node terpisah**.

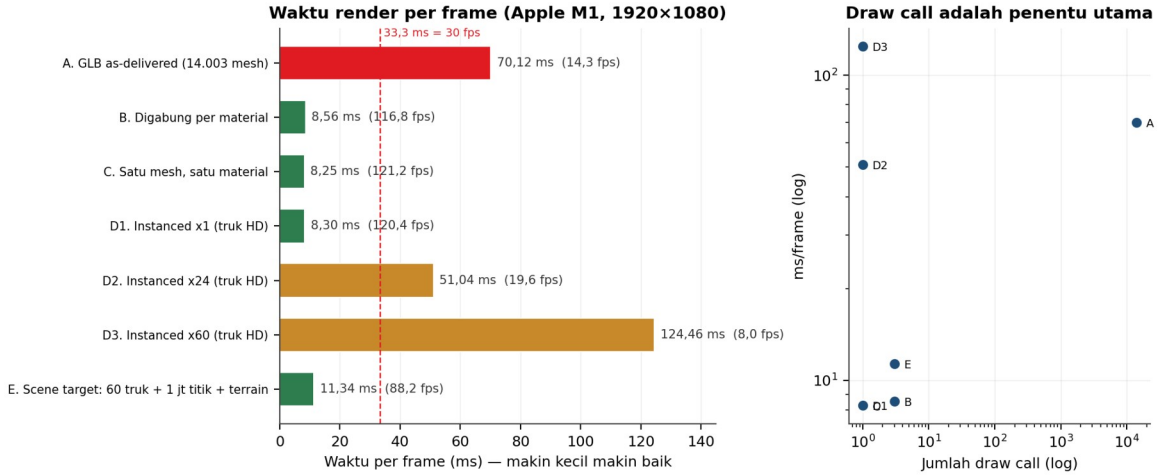
Setiap mesh node menjadi satu draw call. Empat belas ribu draw call berarti CPU harus menyiapkan empat belas ribu perintah gambar untuk setiap frame — dan itu pekerjaan yang harus selesai dalam 16 milidetik bila ingin 60 fps. Hasilnya bisa ditebak, tetapi saya tetap mengukurnya supaya angkanya bisa dipakai:

Tabel 18. Hasil pengukuran — model dump truck yang sama, tiga cara memuat

Skenario	Draw call	Segitiga	ms/frame	fps
A. GLB as-delivered (14.003 mesh)	14.003	1.131.975	70.12	14.3
B. Digabung per material	3	1.131.975	8.56	116.8
C. Satu mesh, satu material	1	1.131.975	8.25	121.2

Baris pertama dan ketiga memuat **geometri yang identik** — 1.131.975 segitiga, material yang sama, tekstur yang sama. Bedanya hanya struktur: yang pertama memuat 14.003 mesh, yang ketiga memuat satu mesh hasil penggabungan. Hasilnya 14,3 fps berbanding 121,2 fps — **8,5 kali lebih cepat tanpa mengurangi satu segitiga pun**.

Dari dua angka itu saya menurunkan biaya per draw call: (70,12 − 8,25) ms dibagi (14.003 − 1) draw call menghasilkan sekitar **4,4 mikrodetik per draw call**. Angka ini kecil per satuannya, dan justru itulah pelajarannya: satu draw call tidak terasa, tetapi empat belas ribu dijumlahkan menjadi 62 milidetik — empat kali lipat anggaran satu frame pada 60 fps.



Gambar 4. Kiri: fps per skenario pada skala logaritmik, dengan garis putus-putus sebagai batas 30 fps. Kanan: waktu per frame. Perhatikan bahwa penurunan tajam terjadi saat jumlah mesh naik, bukan saat jumlah segitiga naik.

4.3 Uji instancing — batas sebenarnya bukan 1 juta segitiga

Setelah struktur dirapikan, pertanyaan berikutnya adalah berapa banyak kendaraan yang bisa ditampilkan. Di tambang, satu scene bisa berisi 40–70 unit. Saya menguji dengan `InstancedMesh`: satu geometri truk, digambar berkali-kali dengan matriks berbeda. Ini cara yang benar untuk armada, karena semua truk HD memang berbagi model yang sama.

Tabel 19. Hasil uji instancing — satu geometri, N instance

Skenario	Draw call	Segitiga dirender	ms/frame	fps
D1. Instanced x1 (truk HD)	1	1.131.975	8.30	120.4
D2. Instanced x24 (truk HD)	1	27.167.400	51.04	19.6
D3. Instanced x60 (truk HD)	1	67.918.500	124.46	8.0

Angka ini memberi dua batas yang sangat berguna. Pada 24 instance (27,2 juta segitiga), masih tercapai 19,6 fps. Pada 60 instance (67,9 juta segitiga), turun ke 8,0 fps. Dari sini throughput vertex dapat dihitung: sekitar 546 juta segitiga/detik pada beban 24 instance, dan 546 juta juga pada beban 60 instance — konsisten, yang berarti pada titik ini penyempitannya sudah berpindah dari CPU ke GPU.

Implikasi praktisnya jelas dan penting untuk anggaran: **menampilkan 60 truk HD berdetail penuh secara serentak tidak mungkin di perangkat kelas ini** (8 fps). Tetapi itu bukan masalah, karena secara operasional juga tidak perlu. Yang perlu adalah 60 truk yang **terlihat benar** dari jarak kamera operasional — dan untuk itu model perlu punya tingkat detail (LOD) rendah. Inilah sebabnya Bab 7 menggambar truk sebagai blok sederhana, bukan model 1,13 juta segitiga.

4.4 Scene target — angka yang saya pakai untuk keputusan

Pengukuran yang paling relevan bukanlah model tunggal, melainkan campuran yang menyerupai layar sungguhan: terrain, ortomosaik, 60 unit armada, dan satu juta titik untuk jejak lintasan. Saya menyusun scene itu dan mengukurnya.

Tabel 20. Scene target realistis — komposisi dan hasil

Skenario	Draw call	Segitiga	Titik	ms/frame	fps
Scene target (60 truk + 1 jt titik + terrain)	3	105.200	1.000.000	11.34	88.2

Hasilnya **88,2 fps** atau 11,34 ms per frame. Angka ini penting karena dua hal. Pertama, ia berada jauh di atas batas 30 fps yang saya tetapkan sebagai anggaran. Kedua, ia dicapai **tanpa** memakai WebGPU, tanpa GPU diskrit, dan tanpa trik dekoratif — hanya WebGL2 biasa pada GPU terintegrasi laptop.

Dengan target 30 fps (33,3 ms per frame), tersisa 21,99 ms atau **66% headroom** untuk pekerjaan lain: memperbarui posisi, menggambar panel UI, dekode data jaringan, dan animasi. Headroom sebesar ini bukan kemewahan — ia adalah ruang untuk fitur yang belum direncanakan, dan dalam proyek nyata ruang itu selalu terpakai.

HASIL — Angka yang saya pakai untuk keputusan

Scene target berjalan **88,2 fps (11,34 ms/frame)** pada GPU terintegrasi kelas laptop. Anggaran yang saya tetapkan: **30 fps minimum** pada perangkat dispatcher, dengan headroom 66%. Untuk tablet lapangan, target diturunkan ke 30 fps dengan LOD lebih kasar dan mode 2D sebagai fallback. Angka ini adalah **batas bawah** — perangkat dengan GPU diskrit akan lebih cepat.

4.5 Pelajaran yang mengubah prioritas pekerjaan

Dari seluruh pengukuran ini, ada empat kesimpulan yang saya bawa ke Bab 6 dan Bab 9. Keempatnya bukan kesimpulan tentang three.js, melainkan tentang **pekerjaan yang harus dilakukan siapa pun yang membangun visualisasi tambang, dengan mesin apa pun**.

- **Pipeline aset lebih menentukan daripada pilihan mesin.** Selisih antara model apa adanya dan model yang dirapikan adalah 8,5 kali. Selisih antar mesin render di kelas yang sama jauh lebih kecil dari itu. Konsekuensinya: pekerjaan pertama yang berharga bukan memilih mesin, tetapi membangun pipeline konversi aset yang disiplin.
- **Angka "jumlah objek" pada brosur hampir selalu menyesatkan.** Yang menentukan adalah berapa draw call, berapa material, dan berapa tekstur — bukan berapa objek.
- **Detail model harus mengikuti jarak kamera, bukan kebanggaan artistik.** Truk berdetail 1,13 juta segitiga hanya masuk akal bila kamera mendekat. Untuk pandangan seluruh pit, LOD rendah sudah cukup dan itulah yang membuat 60 unit mungkin.
- **Sinkronisasi GPU wajib dalam setiap pengukuran render.** Tanpa itu hasilnya bisa salah hingga dua orde besaran, dan keputusan yang dibangun di atasnya akan salah.

Satu hal yang perlu saya nyatakan terus terang: pengukuran ini dilakukan pada satu mesin, dengan satu model, dan satu komposisi scene. Ia cukup untuk menjawab "apakah mungkin" dan "apa penyempitannya", tetapi tidak cukup untuk menjanjikan angka fps pada perangkat tertentu di lapangan. Untuk itu diperlukan uji pada perangkat target yang sebenarnya — dan itu pekerjaan Fase 1, bukan pekerjaan riset ini.

5 KEBUTUHAN DATA DAN METODE INTEGRASI

5.1 Skema kanonik — satu baris, satu sampel

Masalah terbesar yang saya temukan bukanlah kekurangan data, melainkan **keragaman format**. Log RTK keluar sebagai NMEA, telemetri kendaraan keluar sebagai CSV, ortomosaik sebagai GeoTIFF, dan masing-masing punya cara sendiri menyebut waktu, satuan, dan kualitas. Selama formatnya beragam, setiap fitur baru harus menulis pembaca sendiri — dan itu sumber bug yang tak habis.

Karena itu saya mengusulkan satu skema kanonik. Prinsipnya sederhana: **satu baris adalah satu sampel dari satu sumber pada satu waktu**, dengan satuan SI, waktu UTC, dan kualitas yang selalu ikut. Semua adapter bertugas mengubah format asli ke bentuk ini; tidak ada bagian lain dari sistem yang boleh tahu format asli.

```
{
  "ts_utc":        "2026-10-28T10:41:02.113Z",   // waktu sumber, UTC, ISO-8601
  "ts_ingest":     "2026-10-28T10:41:02.488Z",   // waktu diterima gateway
  "source":        "rtk-lite-01",                // identitas perangkat
  "asset_id":      "HD-033",                     // unit yang diukur (boleh null)
  "kind":          "position",                   // position|telemetry|detection|video

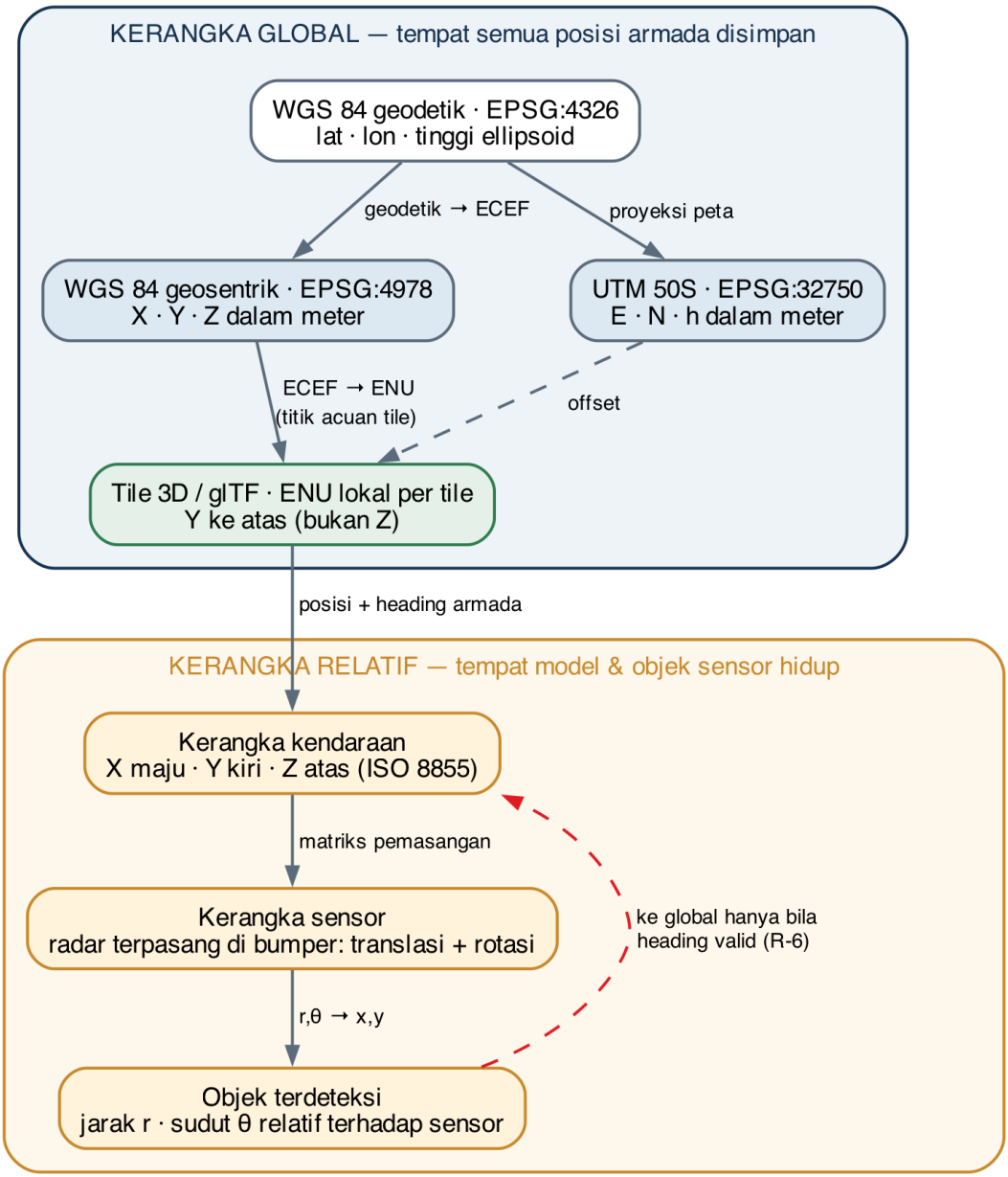
  "pos": {                                        // EPSG:4978, meter
    "ecef":        [-2377291.482, 5397071.113, -2291004.771],
    "crs_src":     "EPSG:4326"
  },
  "quality": {
    "fix":         "FLOAT",                      // NONE|SINGLE|FLOAT|FIX
    "satellites":  41,
    "hdop":        0.44,
    "corr_age_s":  1.2
  },
  "motion": {
    "speed_mps":   7.61,                         // SI, bukan km/h
    "course_deg":  318.4,
    "course_valid": false                        // false bila kecepatan ~0
  },
  "payload": {}                                   // khusus kind=detection / telemetry
}
```

Beberapa keputusan di dalam skema ini perlu penjelasan, karena tampak sepele tetapi berdampak besar:

- **Posisi disimpan sebagai ECEF (EPSG:4978), bukan lat/lon.** Alasannya matematis: interpolasi dan perhitungan jarak pada lat/lon salah kecuali memakai rumus geodetis. Dalam ECEF, semuanya adalah vektor meter biasa. Konversi ke lat/lon hanya dilakukan saat ditampilkan.
- **Dua cap waktu, bukan satu.** `ts_utc` adalah waktu kejadian di sumber; `ts_ingest` adalah waktu data diterima. Selisihnya adalah latensi, dan latensi itulah yang harus ditampilkan kepada pengguna — bukan disembunyikan.
- **course_valid wajib ada.** Ini pelajaran langsung dari data: 1.146 dari 3.601 sampel punya course yang membeku karena kecepatan mendekati nol. Memakai course beku untuk mengarahkan model 3D akan membuat truk menghadap arah yang salah.
- **Kualitas bukan opsional.** Tanpa `fix`, `hdop`, dan `corr_age`, pengguna tidak bisa membedakan posisi yang akurat 2 cm dari posisi yang meleset 3 meter.
- **Satuan SI.** Kecepatan dalam m/s, bukan km/h. Konversi ke km/h hanya untuk tampilan. Ini mencegah kelas bug satuan yang sudah pernah terjadi di proyek sebelumnya.

5.2 Sistem koordinat — tiga kerangka yang tidak boleh dicampur

Ini bagian yang paling sering salah dalam proyek digital twin, dan kesalahannya sering baru terlihat setelah sistem dipakai berbulan-bulan. Ada tiga kerangka koordinat yang hidup bersamaan, dan masing-masing punya aturan sendiri.



Gambar 5. Tiga kerangka koordinat: global geodetik (disimpan), global kartesian (dihitung), dan relatif kendaraan (tempat model dan objek sensor hidup).

Kerangka pertama adalah global geodetik — latitudo, longitudo, tinggi. Ini yang dipakai manusia dan yang keluar dari NMEA. **Kerangka kedua** adalah global kartesian: ECEF (EPSG:4978) untuk perhitungan, atau UTM 50S (EPSG:32750) untuk peta datar. Ortomosaik Minergo berada di UTM 50S, jadi konversi ini nyata dan harus benar. **Kerangka ketiga** adalah relatif kendaraan: tempat model 3D dipasang, dan tempat objek radar dinyatakan.

Aturan yang saya tetapkan, dan yang harus dipatuhi oleh siapa pun yang melanjutkan pekerjaan ini:

11. Posisi armada **disimpan** dalam EPSG:4978. Tidak ada penyimpanan lat/lon di basis data operasional.
12. Model 3D kendaraan **selalu** berada di kerangka kendaraan (X maju, Y kiri, Z atas, ISO 8855), lalu dipasang ke dunia lewat matriks posisi dan heading.
13. Objek radar **selalu** relatif terhadap sensor, bukan terhadap dunia. Mengubahnya menjadi global membutuhkan heading yang valid — dan heading tidak selalu valid (R-6).

14. Ortomosaik dan terrain memakai UTM 50S pada sumbernya, lalu dikonversi ke ENU lokal per tile. Jangan mencampur UTM dengan ECEF secara langsung.
15. Setiap kali ada perbedaan posisi antar sumber, selisihnya **diukur dan dicatat**, tidak dirata-ratakan diam-diam.

Poin terakhir itu bukan teori. Pada data lapangan yang saya periksa, posisi dari log RTK dan posisi dari log telemetri unit berbeda rata-rata **1,27 meter**, dengan selisih maksimum **7,27 meter**. Bila kedua sumber digambar tanpa penjelasan, pengguna akan melihat dua truk untuk satu kendaraan — dan menyimpulkan sistemnya rusak. Padahal yang terjadi adalah dua metode penentuan posisi yang berbeda akurasi. Selisih ini harus ditampilkan sebagai atribut, misalnya "posisi RTK" versus "posisi GNSS unit".

5.3 Data yang sudah ada — hasil inventaris

Ini bagian yang paling melegakan dari seluruh riset. Bahan baku digital twin sudah cukup banyak; yang belum adalah sambungannya.

Tabel 21. Inventaris aset nyata yang ditemukan di folder Minergo

Aset	Properti terukur	Status	Dipakai untuk
Ortomosaik drone	23.249 × 17.296 px · 1,35 cm/px · EPSG:32750 · 7,4 ha · survei 14 Okt 2026	**SIAP**	Permukaan dasar peta 3D
Log RTK (28 Okt 2026)	3.601 titik · 3.602 s · 10,9 km · 32,9% FIX	**SIAP**	Posisi armada, trail, replay
Model dump truck	1.131.975 tri · 14.003 mesh · 23,59 MB GLB (Draco)	**SIAP setelah dirapikan**	Model armada (dengan LOD)
Model excavator	6.452 tri · 26 mesh · 40 KB	**SIAP**	Model alat berat
Telemetri unit (OBU)	CSV per perjalanan · kecepatan, posisi, status	**SEBAGIAN** — belum berskema	Panel unit, analitik
Batas area (shapefile)	252 byte · 0 geometri · WGS 84	**KOSONG**	Geofence (harus dibuat ulang)
DSM / point cloud drone	Tidak ada berkas	**TIDAK ADA**	Terrain 3D sejati
Object list radar	Tidak ada dokumen protokol	**TIDAK ADA**	Use Case A (radar)

5.4 Kualitas data lapangan — apa yang angka katakan

Ini bagian yang paling tidak nyaman, dan karena itu paling penting. Data RTK yang saya periksa adalah rekaman perjalanan nyata selama satu jam. Dari 3.601 sampel, hanya **1.184 (32,9%) berstatus FIX**; sisanya 2.417 sampel (67,1%) berstatus **FLOAT**. Satelit rata-rata 48 dan HDOP 0,40 — angka yang bagus. Jadi masalahnya bukan geometri satelit, melainkan hilangnya solusi integer ambiguity, yang khas terjadi di lingkungan tambang karena multipath dari tebing dan kendaraan besar.

Apa artinya bagi digital twin? FIX berarti akurasi sentimeter. FLOAT berarti akurasi desimeter sampai meter. Untuk menampilkan posisi armada di peta, keduanya cukup. Untuk **menegakkan geofence keselamatan**, FLOAT tidak cukup — truk bisa terlihat di dalam zona padahal berada 2 meter di luarnya. Karena itu kelas akurasi harus tampil di layar, dan aturan keselamatan tidak boleh memakai sampel FLOAT tanpa penanda.

Tabel 22. Statistik log RTK nyata — 3.601 sampel, 28 Oktober 2026

Metrik	Nilai	Implikasi bagi desain
Durasi / jarak	3.602 s / 10.891 m	Cukup untuk satu siklus haul penuh — layak dijadikan data replay contoh
Kecepatan rata-rata / p95 / maks	10,82 / 27,49 / 35,79 km/jam	Kecepatan p95 masih di bawah batas 40 km/jam pada umumnya jalur angkut
Laju sampel	1 Hz (3.601 sampel dalam 3.602 s)	**Penyempit utama** — lihat perhitungan di bawah

Metrik	Nilai	Implikasi bagi desain
Jarak antar sampel pada FIX	3,03 m	Pada 30 fps, posisi hanya berubah sekali setiap 30 frame; interpolasi wajib
Status solusi	32,9% FIX · 67,1% FLOAT	Kelas akurasi harus tampil di UI
Satelit / HDOP	48,4 rata-rata · HDOP 0,40	Geometri satelit baik; FLOAT bukan disebabkan jumlah satelit
Umur koreksi RTCM	1,11 s rata-rata · 10,0 s maksimum	Koreksi sempat putus hingga 10 detik — indikator jaringan perlu ada di UI
Course beku	1.146 dari 3.601 sampel	Heading tidak boleh dipakai saat kecepatan ≈ 0 (risiko R-6)
Checksum NMEA salah	4 dari 3.605 kalimat	Kecil, tetapi harus ditangani — bukan diabaikan

Dari angka ini muncul satu perhitungan yang sangat menentukan pengalaman pengguna. Pada laju 1 Hz dan kecepatan rata-rata 10,8 km/jam (3,0 m/s), posisi truk melompat sekitar **3,03 meter setiap sampel**. Bila data digambar apa adanya, truk akan tampak berpindah secara tersentak sekali per detik. Pada 30 fps, itu berarti 30 frame dengan posisi diam lalu satu lompatan.

Solusinya bukan menaikkan laju sensor lebih dulu — meskipun itu juga perlu (LC29H mendukung maksimum 10 Hz menurut datasheet-nya, sementara konfigurasi sekarang 1 Hz). Solusi yang lebih murah adalah **interpolasi di sisi klien** dengan buffer kecil: tampilkan posisi pada waktu `sekarang − latensi`, dan gerakkan secara halus menuju sampel berikutnya. Ini membuat gerakan terlihat kontinu tanpa menambah bandwidth, dengan konsekuensi bahwa yang terlihat adalah posisi **perkiraan**, bukan posisi terukur terakhir. Karena itu label latensi wajib ada.

5.5 Sinkronisasi waktu

Ketika satu layar menampilkan posisi kendaraan, objek radar, dan video secara bersamaan, ketiganya harus berada pada waktu yang sama. Tanpa itu, replay insiden akan menampilkan objek yang belum ada saat tabrakan terjadi — dan kesimpulan investigasinya salah.

Tabel 23. Kebutuhan presisi waktu per jenis data

Data	Presisi yang dibutuhkan	Alasan	Sarana
Posisi armada	±100 ms	Pada 36 km/jam, 100 ms = 1 m pergeseran	NTP di gateway (cukup)
Objek radar	±10 ms	Objek bergerak cepat; salah waktu = salah posisi relatif	PTP atau 1PPS dari modul GNSS
Video (MDVR)	±40 ms (1 frame)	Untuk membuktikan urutan kejadian	Timestamp JT808 + sinkronisasi berkala
Telemetri mesin	±500 ms	Perubahan RPM/beban relatif lambat	NTP di gateway
Ortomosaik / peta	Tidak relevan	Data statis, ditandai tanggal survei	Metadata tanggal di UI

Modul GNSS yang dipakai Minergo punya keunggulan yang jarang dimanfaatkan: output **1PPS dengan akurasi tipikal 100 ns** menurut datasheet LC29H. Pulsa ini bisa dipakai untuk menyelaraskan pencatatan waktu antar perangkat di satu kendaraan, tanpa memerlukan server PTP. Untuk tahap sekarang ini belum perlu, tetapi penting dicatat sebagai jalur yang sudah tersedia di perangkat yang ada.

5.6 Integrasi radar — di mana persisnya terhenti

Saya perlu menjelaskan bagian ini dengan sangat spesifik, karena inilah perbedaan antara "belum dikerjakan" dan "tidak bisa dikerjakan dengan perangkat ini".

Datasheet STONKAM PSA27 menyebutkan: frekuensi 76–77 GHz FMCW, jangkauan deteksi 0,2–40 m, lebar pemantauan 0–20 m, hingga lima zona yang dapat dikonfigurasi, toleransi ±15 cm, sudut 120° horizontal dan 14° vertikal, proteksi IP69K, dan daya 10–32 VDC. Semua ini adalah spesifikasi **deteksi**.

Yang tidak ada dalam dokumen itu adalah spesifikasi **pelaporan**: format pesan objek, laju pembaruan objek, jumlah target maksimum, dan struktur protokol. Antarmuka yang disebut hanyalah konfigurasi (USB-to-UART atau Wi-Fi), satu input trigger, dan satu keluaran alarm.

CISBO C95M lebih singkat lagi: 77 GHz, jangkauan 5 m × 6 m, FOV 180° horizontal, daya 9–36 VDC, dan antarmuka berupa **LED dan buzzer 80–100 dB**. Tidak ada klaim keluaran data.

Kesimpulannya bukan bahwa perangkat ini buruk. Untuk fungsi aslinya — memperingatkan pengemudi akan halangan saat mundur — alarm adalah keluaran yang tepat, murah, dan andal. Masalahnya adalah **jenis keluaran yang berbeda dari yang dibutuhkan visualisasi**. Alarm bersifat biner: ada halangan, atau tidak. Visualisasi membutuhkan vektor: di mana, sejauh apa, pada sudut berapa.

Tabel 24. Perbandingan kontrak data — yang dibutuhkan vs yang tersedia

Yang dibutuhkan visualisasi	PSA27	C95M	Radar kelas object-list
Jarak objek (m)	Tidak tersedia	Tidak tersedia	Ya
Sudut azimut (°)	Tidak tersedia	Tidak tersedia	Ya
Kecepatan relatif	Tidak tersedia	Tidak tersedia	Ya
Jumlah objek bersamaan	Tidak tersedia	Tidak tersedia	Ya (biasanya 20–64)
Laju pembaruan objek	Tidak tersedia	Tidak tersedia	Ya (10–20 Hz)
Alarm biner / zona	**Ya**	**Ya**	Ya
Dokumen protokol publik	**TIDAK DITEMUKAN**	**TIDAK DITEMUKAN**	Sebagian (NDA)
Antarmuka data	USB-UART (konfigurasi), alarm	LED + buzzer	CAN FD / Ethernet / UART

KEPUTUSAN — Batas yang saya tolak untuk dilewati

Saya bisa saja menggambar panel radar yang tampak meyakinkan di wireframe — beberapa titik oranye bergerak di sekitar truk — dan tidak ada yang akan tahu bedanya sampai integrasi sebenarnya dimulai. Saya memilih tidak melakukannya. Panel radar pada wireframe Bab 7 digambar **kosong dengan keterangan** alasan ketidaktersediaannya, dan itu justru menyampaikan informasi yang lebih berguna kepada pengambil keputusan daripada animasi palsu: bahwa ada pekerjaan pengadaan atau negosiasi vendor yang harus diselesaikan lebih dulu.

5.7 Kontrak data yang harus diminta ke vendor

Bila jalur yang dipilih adalah tetap memakai perangkat yang ada, berikut adalah spesifikasi minimum yang harus diminta. Saya tuliskan dalam bentuk yang bisa langsung dikirim sebagai surat, karena dokumen ini akan dipakai untuk itu.

```
Permintaan informasi teknis — integrasi data radar ke sistem visualisasi

Untuk setiap model radar yang kami gunakan, kami memerlukan:

 1. Daftar perintah/protokol untuk membaca OBJECT LIST (bukan hanya status alarm),
    termasuk format frame, checksum, dan contoh byte stream nyata.

 2. Spesifikasi object list:
      - jumlah target maksimum per frame
      - laju pembaruan (Hz)
      - besaran yang dilaporkan per target (jarak, azimut, elevasi,
        kecepatan relatif, RCS, ID pelacakan)
      - resolusi dan akurasi masing-masing besaran

 3. Apakah tersedia SDK, driver, atau contoh kode (C/C++/Python)?
    Apakah tersedia di bawah NDA? Berapa lama prosesnya?

 4. Antarmuka fisik yang tersedia untuk data (bukan konfigurasi):
    UART / CAN / CAN FD / Ethernet? Pinout dan level tegangan?

 5. Konfirmasi jangkauan dan FOV untuk model yang kami miliki, dalam dokumen
    resmi bertanda tangan — bukan halaman pemasaran.

 6. Apakah perangkat dapat mengeluarkan data mentah (raw ADC/cube) selain
    object list? Bila ya, dengan antarmuka apa dan laju berapa?
```

6 ARSITEKTUR YANG DIUSULKAN

6.1 Prinsip perancangan

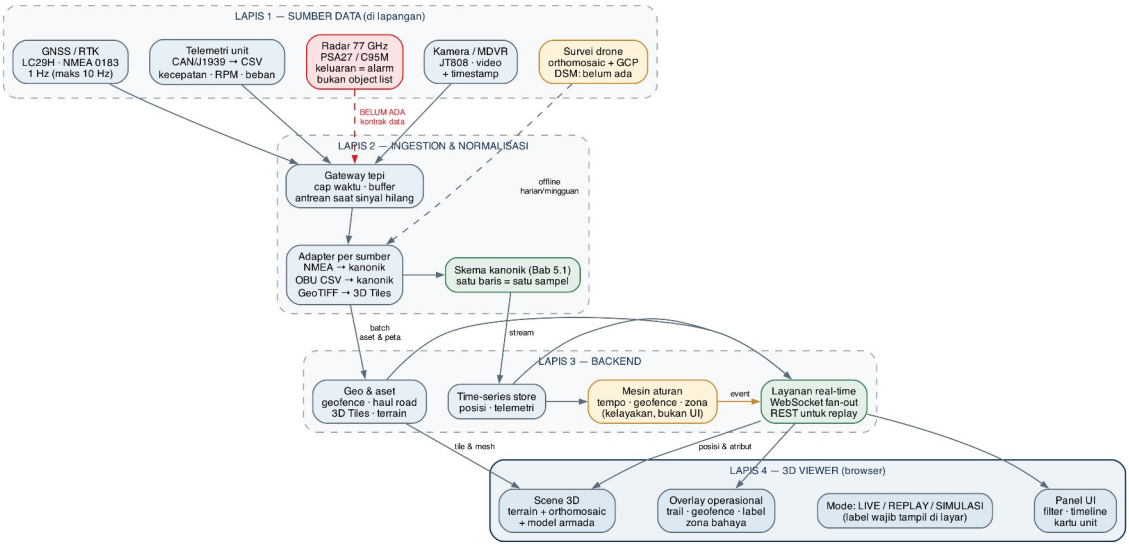
Arsitektur ini saya susun dengan lima prinsip yang lahir langsung dari temuan sebelumnya — bukan dari pola umum. Setiap prinsip menjawab satu masalah yang sudah terbukti ada, bukan masalah yang dibayangkan.

Tabel 25. Prinsip arsitektur dan masalah nyata yang diaturnya

Prinsip	Masalah yang dijawab	Konsekuensi konkret
Pisahkan bidang data dari bidang tampilan	67,1% sampel RTK berstatus FLOAT, dan laju 1 Hz; tampilan tidak boleh mengikuti data mentah apa adanya	Ada lapisan normalisasi dan buffer; klien menerima data yang sudah berskema
Satu skema kanonik, banyak adapter	Format sumber beragam: NMEA, CSV, GeoTIFF, JSON	Setiap sumber punya adapter kecil; tidak ada logika format di dalam viewer
Tandai ketidakpastian, jangan sembunyikan	Kelas akurasi berbeda jauh antar sampel	Kualitas adalah field wajib; UI menampilkannya; trail diberi warna menurut kualitas
Mode adalah status, bukan setelan	Risiko pengguna salah membaca live sebagai replay atau sebaliknya	Indikator mode selalu tampil, tidak bisa disembunyikan
Pekerjaan berat di server, bukan di klien	Tablet lapangan dengan 4 GB RAM	Tiling, LOD, dan agregasi dilakukan sekali di server, bukan per klien

6.2 Diagram arsitektur

Diagram berikut menunjukkan empat lapis sistem, dari sensor di lapangan sampai viewer di browser. Yang perlu diperhatikan adalah dua garis putus-putus merah: keduanya menandai sambungan yang **belum ada kontrak datanya**, dan sengaja digambar begitu supaya celahnya terlihat, bukan tersembunyi.



Gambar 6. Arsitektur empat lapis. Kotak merah (radar) adalah satu-satunya sumber yang belum punya kontrak data; kotak kuning (survei drone) adalah sumber batch, bukan streaming.

6.3 Penjelasan setiap lapis

Lapis 1 — sumber data

Lima sumber, dengan sifat yang berbeda dan karena itu harus diperlakukan berbeda. **GNSS/RTK** mengeluarkan NMEA pada 1 Hz. **Telemetri unit** keluar sebagai CSV per perjalanan — bukan

streaming, melainkan berkas yang diunduh setelah perjalanan selesai. **Kamera/MDVR** mengeluarkan video dengan timestamp JT808. **Radar** mengeluarkan alarm biner. **Survei drone** menghasilkan ortomosaik yang diperbarui berkala, bukan setiap detik.

Perbedaan sifat ini penting: tiga sumber bersifat streaming (GNSS, kamera, radar bila kelak ada), satu sumber semi-batch (telemetri), dan satu sumber batch (drone). Menyamakan perlakuan terhadap kelimanya akan menghasilkan arsitektur yang boros.

Lapis 2 — ingestion dan normalisasi

Gateway tepi bertugas tiga hal yang tidak boleh diserahkan ke server pusat: memberi cap waktu saat data **tiba** (bukan saat diproses), menyangga data ketika jaringan hilang, dan mengirim ulang secara berurutan ketika jaringan kembali. Di tambang, jaringan hilang adalah keadaan normal, bukan pengecualian.

Adapter bertugas mengubah format asli menjadi skema kanonik Bab 5.1. Satu adapter per sumber, masing-masing kecil dan dapat diuji sendiri. Ini pekerjaan yang tampak membosankan tetapi menentukan: bila adapter benar, seluruh sistem di atasnya menjadi sederhana; bila adapter bocor, kompleksitasnya menyebar ke mana-mana.

Lapis 3 — backend

Empat komponen dengan tanggung jawab yang sengaja dipisah. **Penyimpanan deret waktu** menyimpan posisi dan telemetri — ini yang membuat replay mungkin. **Penyimpanan geo dan aset** menyimpan geofence, geometri haul road, dan tile 3D. **Mesin aturan** mengevaluasi kondisi (kecepatan melebihi batas di dalam zona, keluar geofence, idle terlalu lama) dan mengeluarkan event. **Layanan real-time** menyebarkan data ke klien lewat WebSocket, dan melayani query replay lewat REST.

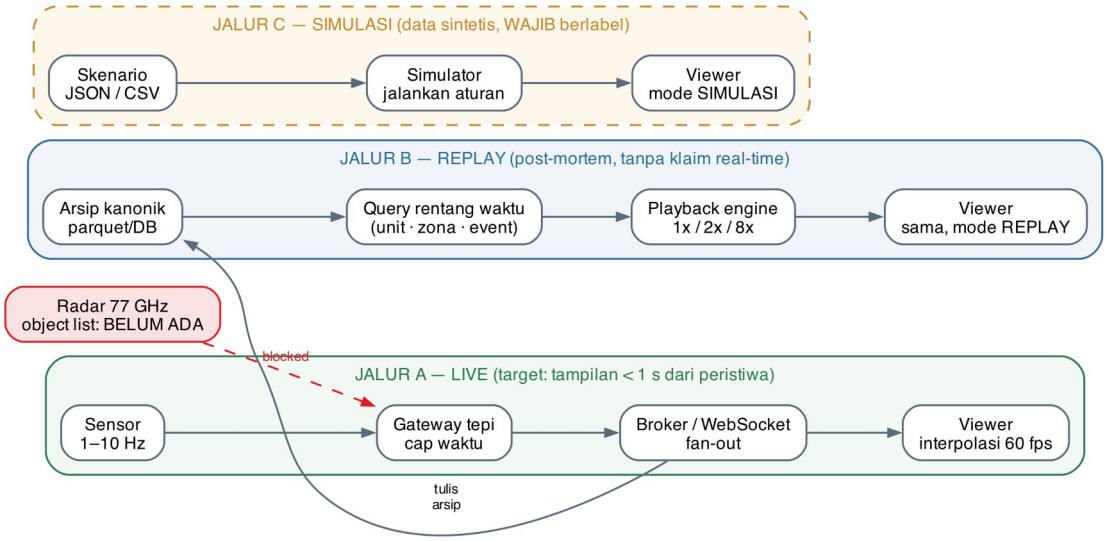
Pemisahan mesin aturan dari layanan real-time bukan formalitas arsitektur. Alasannya operasional: aturan keselamatan berubah lebih sering daripada kode penyajian, dan aturan yang salah harus bisa diperbaiki tanpa menyentuh jalur data. Selain itu, mesin aturan bisa diuji dengan data historis — "apakah aturan baru ini akan menandai kejadian bulan lalu?" — dan itu kemampuan yang sangat berharga untuk validasi.

Lapis 4 — viewer 3D

Viewer adalah lapisan yang paling terlihat, tetapi dalam arsitektur ini justru yang paling tipis logikanya. Ia menerima tile dari server, posisi dari WebSocket, dan menggambar. Semua kecerdasan ada di lapis 2 dan 3. Prinsip ini menjaga viewer tetap bisa diganti — dan pada kenyataannya ia **akan** diganti, karena teknologi render berubah lebih cepat daripada skema data.

6.4 Tiga jalur data: live, replay, simulasi

Ketiga mode di Bab 2.6 bukan tiga tampilan, melainkan tiga jalur data yang berbeda. Menggambarnya terpisah membuat konsekuensi teknisnya terlihat jelas.



Gambar 7. Tiga jalur data. Jalur LIVE dan REPLAY berbagi viewer yang sama; jalur SIMULASI memakai viewer yang sama tetapi sumber datanya skenario buatan, dan wajib berlabel.

Perhatikan bahwa jalur LIVE menulis ke arsip. Ini keputusan yang disengaja: replay tidak dibangun sebagai fitur terpisah, melainkan sebagai **konsekuensi** dari penyimpanan yang benar. Bila jalur live menyimpan data kanonik secara konsisten, replay hampir gratis. Bila tidak, replay harus dibangun ulang dengan cara khusus — dan itulah yang biasanya terjadi pada proyek yang tidak merancangnya dari awal.

6.5 Di mana komponen diletakkan (on-premise)

Satu persyaratan yang saya anggap tidak bisa dinegosiasikan: **sistem harus berjalan tanpa internet publik.** Alasannya bukan paranoia, melainkan kenyataan operasional — jaringan tambang sering terputus, dan koordinat serta data produksi tidak seharusnya keluar dari jaringan perusahaan. Ini juga risiko R-8.

Tabel 26. Penempatan komponen dan ketergantungan pada internet

Komponen	Lokasi	Butuh internet publik?	Catatan
Gateway tepi	Di site, dekat perangkat	Tidak	Harus tahan putus jaringan
Backend & basis data	Server on-premise	Tidak	Akses via VPN untuk tim pusat
Tile 3D & ortomosaik	Server file on-premise	Tidak	Di-generate sekali di server
Viewer (browser)	Disajikan dari server lokal	Tidak	Tanpa CDN, tanpa token
Peta dasar (bila perlu)	Tile lokal (self-host)	Tidak	MapLibre + tile sendiri
Font / pustaka JS	Di-bundle, bukan CDN	Tidak	Build statis

Poin terakhir terlihat sepele tetapi sering menjatuhkan sistem di lapangan: aplikasi yang memuat font atau pustaka dari CDN akan kehilangan tampilannya ketika internet putus, dan gejalanya sulit didiagnosis karena kodenya benar. Semua dependensi harus di-bundle dalam build.

6.6 Yang sengaja tidak saya masukkan ke arsitektur

Sama pentingnya dengan apa yang ada: beberapa hal populer yang sengaja saya tinggalkan, dengan alasan.

- **Microservices.** Untuk beban satu tambang dengan puluhan unit, empat komponen yang disebut di atas cukup sebagai satu atau dua proses. Memecahnya menjadi belasan layanan menambah biaya operasional tanpa manfaat.

- **Kubernetes.** Satu server on-premise tidak membutuhkan orkestrasi klaster. Docker Compose sudah memadai dan jauh lebih mudah dirawat oleh tim kecil.
- **Event streaming platform (Kafka dan sejenisnya).** Untuk 60 unit pada 1–10 Hz, yaitu ratusan pesan per detik, broker ringan atau bahkan WebSocket langsung sudah cukup. Kafka berguna pada skala ribuan perangkat.
- **WebGPU sebagai syarat.** Dukungan 86,83% berarti 13% perangkat gagal, dan itu terlalu banyak untuk perangkat kantor. WebGL2 dulu, WebGPU sebagai peningkatan.
- **Digital twin permukaan (cut/fill).** Menggiurkan dan sering diminta, tetapi butuh DSM/point cloud yang belum ada. Memasukkannya ke arsitektur sekarang berarti merancang untuk data yang tidak dimiliki.

Daftar ini bukan penolakan terhadap teknologi tersebut. Ini penundaan yang sadar: setiap komponen yang tidak dimasukkan adalah satu hal yang tidak perlu dirawat, diperbarui, dan dipelajari oleh tim yang kecil. Arsitektur yang baik untuk Minergo bukan arsitektur yang paling lengkap, melainkan yang paling mudah dioperasikan oleh tiga orang.

7 KONSEP TAMPILAN (WIREFRAME)

7.1 Dasar perancangan

Wireframe ini bukan gambar hiasan sebelum aplikasi dibuat. Ia adalah alat uji keputusan: dengan menggambar layar secara spesifik, menjadi jelas informasi apa yang harus tersedia, siapa yang harus menyediakannya, dan apa yang terjadi bila tidak ada. Beberapa keputusan di Bab 5 dan Bab 6 lahir justru dari proses menggambar ini — misalnya keharusan menampilkan kelas akurasi, dan keputusan menampilkan panel radar dalam keadaan kosong.

Tiga aturan yang saya pegang dalam menggambar:

16. **Setiap angka di layar harus punya sumber.** Bila sebuah angka tidak bisa dihitung dari data yang ada, ia tidak boleh muncul. Itulah sebabnya panel radar kosong.
17. **Ketidakpastian harus terlihat, bukan tersembunyi.** Warna trail menunjukkan kualitas posisi; label latensi selalu tampil; mode (LIVE/REPLAY/SIMULASI) tidak bisa disembunyikan.
18. **Ponsel bukan versi kecil dari desktop.** Pada layar kecil, kolom kanan tidak dilipat menjadi ikon-ikon kecil — ia diganti dengan lembar bawah yang muncul saat unit dipilih. Prioritasnya peta, bukan panel.

PERINGATAN — Apa yang sedang dilihat pembaca

Gambar pada bab ini adalah **rancangan**, bukan tangkapan layar aplikasi yang sudah jadi. Angka yang muncul di dalamnya diambil dari data lapangan nyata: RTK 28 Oktober 2026 (kecepatan 27,4 km/h, 41 satelit, HDOP 0,44, status FLOAT, course beku 1,1 s), ortomosaik 14 Oktober 2026 (1,35 cm/px), dan hasil uji render Lampiran C (88 fps, 3 draw call). Aplikasinya belum dibangun — itu di luar lingkup riset ini.

7.2 Layar desktop — mode LIVE

Tata letak desktop mengikuti satu prinsip: **peta adalah raja**. Viewport 3D menempati sekitar 61% lebar layar, dan semua panel lain adalah pendukung yang bisa dipindai dalam satu pandangan tanpa mengalihkan perhatian dari peta.



Gambar 8. Wireframe desktop 1920×1080, mode LIVE. Panel kiri: navigasi dan lapisan. Tengah: viewport 3D dengan armada, geofence, dan tooltip unit. Kanan: ringkasan armada, peringatan, panel radar (kosong), dan performa. Bawah: timeline 30 menit dengan penanda kejadian.

Keputusan desain pada layar desktop

Tabel 27. Keputusan desain desktop dan alasannya

Elemen	Keputusan	Alasan
Tab mode	LIVE berwarna merah, selalu di kiri atas; tidak bisa disembunyikan	Mencegah kesalahan paling mahal: mengira melihat keadaan sekarang padahal bukan
Indikator latensi	• `LATENCY 840 ms` di bar atas, dengan titik hijau/kuning/merah	Angka konkret lebih jujur daripada ikon "terhubung"
Warna trail	Hijau untuk FIX, kuning untuk FLOAT	Pengguna melihat sendiri di mana posisi bisa dipercaya, tanpa membaca tabel
Label unit	Kode unit + status kualitas + kecepatan, dalam satu baris	Menjawab "siapa itu dan bisakah dipercaya" tanpa klik
Penanda masalah	Lingkaran merah pada unit yang melanggar geofence, dengan teks di bawahnya	Tidak bergantung pada warna saja — penting untuk pengguna buta warna
Panel radar	Kosong, dengan keterangan mengapa	Kejujuran lebih berguna daripada animasi yang tidak bisa diwujudkan
Timeline	Segmen berwarna menurut kualitas RTK	Memperlihatkan bahwa kualitas berubah sepanjang waktu, bukan seragam
Skala & kompas	Keduanya kecil di sudut, tidak mencolok	Dibutuhkan untuk orientasi, tetapi tidak boleh bersaing dengan data

Satu elemen yang mungkin terlihat aneh: panel **PERFORMA VIEWER** di kanan bawah menampilkan "88 fps · 3 draw call · 11,3 ms/frame". Menampilkan metrik teknis kepada dispatcher memang tidak lazim. Saya memasukkannya karena dua alasan: pertama, ia memungkinkan pengguna melaporkan masalah kinerja dengan angka, bukan dengan kalimat "aplikasinya lambat"; kedua, ia adalah pengingat permanen bagi tim pengembang bahwa anggaran render itu nyata dan terukur.

7.3 Layar tablet — mode LIVE

Untuk perangkat lapangan, tata letaknya berubah secara mendasar. Bukan karena layarnya kecil, tetapi karena **situasinya berbeda**: pengawas berdiri di luar, sering dengan sarung tangan, dan tidak sedang menganalisis — ia sedang memverifikasi satu hal tentang satu unit.



Gambar 9. Wireframe tablet/telepon, mode LIVE. Peta mengisi hampir seluruh layar; detail unit muncul sebagai lembar bawah saat unit disentuh, dengan empat metrik besar dan dua aksi.

Tabel 28. Perbedaan keputusan desain desktop vs perangkat lapangan

Aspek	Desktop	Tablet / telepon	Alasan perbedaan
Porsi peta	61% lebar	≈93% lebar	Lapangan: peta adalah satu-satunya hal yang penting
Detail unit	Panel kanan permanen	Lembar bawah saat dipilih	Tidak ada ruang untuk panel permanen
Jumlah metrik	Lima baris + tabel	Empat angka besar	Dibaca sambil berdiri, sering di bawah matahari
Ukuran sentuh	Klik presisi	Target ≥ 44 px	Sarung tangan dan gerakan tangan
Timeline	Penuh dengan penanda	Ringkas, hanya posisi waktu	Navigasi replay dilakukan di tempat duduk, bukan di lapangan
Panel radar	Ada (kosong, dengan keterangan)	Tidak ada	Tidak berguna bila tidak ada data; menghemat ruang

7.4 Hierarki informasi

Satu pertanyaan yang saya ajukan pada diri sendiri saat menggambar: bila pengguna hanya melihat layar selama tiga detik, apa yang harus ia dapat? Jawabannya menentukan urutan prioritas visual. Saya menyusunnya menjadi tiga lapis.

Tabel 29. Hierarki informasi — tiga lapis perhatian

Lapis	Isi	Waktu baca	Perlakuan visual
1 — Sekilas	Mode (LIVE/REPLAY), jumlah unit aktif, ada/tidak peringatan, lokasi unit bermasalah	< 1 detik	Warna jenuh, ukuran besar, posisi tetap
2 — Pemindaian	Posisi & heading tiap unit, status kualitas, kecepatan, batas geofence	2–5 detik	Ukuran sedang, warna terkendali
3 — Pendalaman	HDOP, jumlah satelit, umur koreksi, riwayat lintasan, detail telemetri	> 5 detik, perlu klik	Abu-abu, ukuran kecil, di panel

Kesalahan yang paling sering terjadi dalam dashboard operasional adalah menempatkan informasi lapis 3 di area lapis 1 — misalnya menampilkan HDOP dengan huruf besar di tengah layar. Informasi itu penting, tetapi bukan untuk tiga detik pertama. Ia harus tersedia, bukan menonjol.

7.5 Elemen yang sengaja tidak digambar

Beberapa fitur yang sering muncul dalam demo digital twin sengaja tidak saya masukkan, dan alasannya perlu dicatat supaya tidak dianggap kelalaian.

- **Animasi roda berputar dan suspensi.** Menghabiskan anggaran render untuk hal yang tidak menambah keputusan apa pun. Pada pandangan operasional (kamera jauh), detail ini tidak terlihat.
- **Efek bayangan real-time dan refleksi.** Indah untuk pemasaran, mahal untuk perangkat lapangan. Bayangan statis sederhana sudah cukup untuk memberi kesan kedalaman.
- **Panah animasi arus lalu lintas.** Terlihat mengesankan di presentasi, tetapi menutupi data sebenarnya. Bila ingin menunjukkan arah, gunakan heading unit yang nyata.
- **Panel radar berisi titik objek.** Inilah yang paling sering diminta, dan justru yang paling tidak bisa dipertanggungjawabkan dengan perangkat saat ini.
- **Grafik 3D batang untuk KPI.** Tiga dimensi pada grafik batang selalu mengurangi keterbacaan dibanding dua dimensi. Untuk KPI, gunakan grafik 2D.

Ada satu benang merah dalam daftar ini: semua yang saya tinggalkan adalah hal yang **menambah kesan** tanpa menambah **informasi**. Dalam sistem operasional, kesan yang berlebihan justru berbahaya karena membuat orang percaya pada tampilan lebih dari yang seharusnya.

8 ANALISIS KELAYAKAN DAN RISIKO

8.1 Kerangka penilaian

Kelayakan saya nilai pada lima dimensi yang terpisah, karena sebuah proyek bisa layak secara teknis tetapi tidak layak secara operasional — dan itu kejadian yang sangat umum. Menilai kelima dimensi secara terpisah memaksa saya menyebutkan di mana sebenarnya kekuatannya dan di mana kelemahannya.

Tabel 30. Penilaian kelayakan per dimensi

Dimensi	Penilaian	Dasar	Catatan
Teknis	LAYAK	Scene target 88,2 fps pada GPU terintegrasi; seluruh format aset standar (glTF, 3D Tiles, COG); pustaka yang dipakai berlisensi bebas	Satu-satunya komponen yang tidak layak secara teknis adalah data radar — dan itu masalah kontrak data, bukan kemampuan render
Data	LAYAK SEBAGIAN	Ortomosaik, RTK, model 3D, dan telemetri tersedia dan terukur. DSM, point cloud, geofence, dan object list radar tidak ada	Cukup untuk memulai; tidak cukup untuk seluruh use case
Operasional	LAYAK dengan syarat	Data 1 Hz dan 67,1% FLOAT membatasi ketepatan; interpolasi + label kualitas membuatnya dapat dipakai untuk pemantauan, bukan untuk keputusan hukum	Perlu kesepakatan tertulis tentang apa yang boleh disimpulkan dari tampilan
Biaya	LAYAK	Seluruh pustaka inti gratis. Biaya utama adalah jam-orang dan satu server on-premise	Risiko biaya ada pada layanan hosted (Cesium ion, Mapbox) — dihindari dengan self-host
SDM & perawatan	PERLU PERHATIAN	Butuh kompetensi web 3D + geospasial + backend data. Tim kecil bisa, tetapi menjadi bottleneck bila hanya satu orang	Risiko kebergantungan pada satu orang (bus factor) harus dikelola

8.2 Apa yang layak dikerjakan sekarang

Berdasarkan penilaian di atas, ada pekerjaan yang bisa dimulai tanpa menunggu keputusan atau pengadaan apa pun. Ini penting: studi kelayakan yang berakhir dengan "tunggu" adalah studi yang gagal.

Tabel 31. Pekerjaan yang layak dimulai sekarang, tanpa prasyarat

Pekerjaan	Prasyarat	Keluaran	Risiko
Konversi ortomosaik ke tile yang bisa di-stream	Tidak ada — berkas sudah ada	Peta dasar 3D yang bisa dibuka di browser	Rendah
Pipeline aset model 3D (merge + LOD + Draco/KTX2)	Tidak ada — model sudah ada	Model armada yang ringan dan siap instancing	Rendah
Skema kanonik + adapter NMEA/CSV	Tidak ada	Data lapangan yang seragam dan bisa diputar ulang	Rendah
Viewer armada dasar (posisi + heading + trail)	Tiga hal di atas	Demo UC-1 yang bisa dinilai pengguna	Rendah
Pembuatan geofence dari citra	Butuh waktu surveyor, bukan pengadaan	Poligon zona yang bisa dipakai aturan	Rendah
Uji beban pada perangkat target sebenarnya	Satu laptop & satu tablet milik perusahaan	Angka fps nyata di lapangan	Rendah

8.3 Apa yang belum layak, dan prasyaratnya

Sebaliknya, ada pekerjaan yang tampak paling menarik tetapi harus menunggu. Menuliskan prasyaratnya membuat penundaan ini dapat ditindaklanjuti, bukan sekadar ditunda.

Tabel 32. Pekerjaan yang belum layak dan prasyarat yang harus dipenuhi

Pekerjaan	Mengapa belum layak	Prasyarat yang harus selesai lebih dulu
Objek terdeteksi radar (UC-4)	Perangkat yang ada hanya mengeluarkan alarm biner; tidak ada object list	Dokumen protokol / SDK dari vendor (V-1 s.d. V-4), **atau** pengadaan unit ber-object-list
Twin permukaan / cut-fill (UC-6)	Tidak ada DSM maupun point cloud; ortomosaik hanya 2D	Survei drone dengan overlap memadai + pengolahan ke LAS/COPC (V-6)
Simulasi skenario (UC-5)	Belum ada model aturan maupun data historis yang cukup untuk kalibrasi	Skema kanonik berjalan minimal 3 bulan agar ada data pembanding
Peringatan keselamatan otomatis berbasis posisi	67,1% sampel FLOAT; kesalahan posisi bisa > 1 m	Kenaikan fix rate (perbaikan lokasi antena/base) **atau** kesepakatan bahwa peringatan bersifat indikatif
Multi-site / globe	Belum ada kebutuhan yang terverifikasi; menambah kompleksitas signifikan	Keputusan bisnis, bukan keputusan teknis

8.4 Risiko — daftar dan penilaian

Saya menilai risiko dengan perkalian dampak dan kemungkinan (skala 1–3), sehingga skor 9 adalah yang tertinggi. Yang penting bukan angkanya, melainkan **mitigasinya**: risiko yang tidak punya mitigasi jelas seharusnya menghentikan proyek, bukan hanya dicatat.

Tabel 33. Daftar risiko teknis dan operasional

ID	Risiko	Dampak	Kemungkinan	Skor	Mitigasi
R-1	Kontrak data radar tidak tersedia (hanya alarm LED/buzzer)	Tinggi	Tinggi	9	Minta SDK/protokol ke vendor; siapkan jalur mundur: radar diganti unit ber-object-list
R-2	Draw call meledak karena model tidak dioptimasi	Tinggi	Tinggi	9	Pipeline optimasi wajib: merge per material + InstancedMesh (terbukti 8x)
R-3	Akurasi RTK tidak konsisten (67,1% FLOAT pada log nyata)	Sedang	Tinggi	6	Tampilkan kelas akurasi di UI; jangan jadikan FLOAT dasar keputusan geofence kritis
R-4	Sinkronisasi waktu 1 Hz membuat posisi melompat 3 m	Sedang	Tinggi	6	Interpolasi + timestamp sumber; target 5–10 Hz atau dead-reckoning IMU
R-5	Latensi ingestion > 2 s membuat tampilan tidak dapat dipakai untuk peringatan	Sedang	Sedang	4	Pisahkan jalur peringatan (edge) dari jalur visualisasi (server)
R-6	Lisensi Cesium ion / Mapbox membengkak saat pengguna bertambah	Sedang	Sedang	4	Self-host aset (3D Tiles statis) + MapLibre/terrain lokal
R-7	Ketergantungan vendor engine (Unity) mengunci tim	Sedang	Sedang	4	Pilih stack web terbuka lebih dulu; Unity hanya bila butuh desktop berat

ID	Risiko	Dampak	Kemungkinan	Skor	Mitigasi
R-8	Keamanan data: koordinat & produksi keluar dari jaringan perusahaan	Tinggi	Sedang	6	On-premise, tanpa CDN publik, tanpa token pihak ketiga di klien
R-9	Hardware klien di lapangan (tablet) tidak mampu render	Sedang	Sedang	4	Anggaran render 30 fps, LOD, mode 2D fallback
R-10	Data drone tidak rutin diperbarui → twin cepat kedaluwarsa	Sedang	Tinggi	6	Jadwal survei + tanggal data ditampilkan di UI

8.5 Dua risiko yang paling menentukan

Dari sepuluh risiko di atas, dua mendapat skor tertinggi (9) dan keduanya layak dibahas lebih panjang karena sifatnya berbeda: satu berasal dari luar kendali tim, satu sepenuhnya di dalam kendali tim.

R-1 — Kontrak data radar tidak tersedia

Ini risiko eksternal. Mitigasinya bukan pekerjaan teknis, melainkan pengadaan dan negosiasi. Yang bisa dilakukan tim R&D sekarang hanyalah menyiapkan surat permintaan (Bab 5.7) dan menyiapkan jalur mundur: bila vendor tidak bisa menyediakan object list, pilihannya adalah mengganti unit radar dengan kelas yang menyediakannya — dan itu keputusan manajemen, dengan konsekuensi biaya pengadaan dan sertifikasi ulang.

Yang tidak boleh dilakukan adalah menganggap risiko ini akan hilang sendiri setelah integrasi dimulai. Pengalaman umum menunjukkan sebaliknya: tanpa dokumen protokol, tim akan menghabiskan berminggu-minggu mencoba menebak format melalui eksperimen, dan sering tidak berhasil karena datanya memang tidak dikeluarkan.

R-2 — Draw call meledak karena model tidak dioptimasi

Ini risiko internal, dan justru karena itu paling bisa dikendalikan — sekaligus paling sering terjadi. Angka pengukurannya sudah saya dapat: 14.003 draw call menghasilkan 14,3 fps, sementara 3 draw call menghasilkan 116,8 fps untuk geometri yang sama. Selisihnya 8,5 kali.

Mitigasinya bukan "hati-hati saat mengimpor model", melainkan **pipeline yang memaksa**. Setiap aset yang masuk harus melewati pemeriksaan otomatis: berapa draw call, berapa material, berapa tekstur, berapa segitiga. Aset yang melewati ambang ditolak secara otomatis, bukan diberi peringatan yang bisa diabaikan. Ini keputusan proses, dan nilainya lebih besar daripada keputusan teknologi mana pun dalam dokumen ini.

TEMUAN — Mengapa R-2 lebih penting daripada pilihan mesin

Selisih kinerja antara mesin render yang berbeda di kelas yang sama biasanya berada dalam kisaran 1,5–2 kali. Selisih antara aset yang dioptimasi dan tidak dioptimasi yang saya ukur adalah **8,5 kali**. Artinya: tim yang memilih mesin terbaik tetapi mengabaikan pipeline aset akan kalah dari tim yang memilih mesin biasa tetapi membangun pipeline yang disiplin. Ini temuan yang saya anggap paling berharga dari seluruh riset ini.

8.6 Gap teknis yang harus ditutup

Tabel 34. Gap teknis terverifikasi dan cara menutupnya

ID	Gap	Dampak bila tidak ditutup	Cara menutup
G-1	Berkas buffer dumptruck_edit.bin (108 MB) hilang	Model .gltf tidak bisa dimuat; pakai .glb	Ekspor ulang dari .blend di ~/Documents/HD_3D/dumptruck_edit.blend
G-2	Belum ada DSM / point cloud hasil drone	Twin masih datar (orthomosaic 2D)	Terbangkan ulang dengan overlap 75/65 dan olah ke LAS/COPC
G-3	Kontrak data radar (SDK/protokol) tidak ada	UC-4 tidak bisa dimulai	Surat permintaan SDK ke STONKAM/CISBO; atau uji unit ber-object-list
G-4	Log telemetri belum berskema seragam	Replay & analitik sulit	Definisikan skema kanonik (Bab 5.1) + adapter per sumber
G-5	Belum ada acuan waktu tunggal (PTP/NTP) di unit	Sinkronisasi multi-sensor tidak ketat	Aktifkan NTP/PTP di gateway; cap waktu di titik masuk
G-6	Heading armada tidak stabil (1146 dari 3601 course beku)	Orientasi model 3D bisa salah arah	Fusi course + IMU; jangan pakai course saat kecepatan ~0

8.7 Kesimpulan kelayakan

Digital twin 3D untuk Minergo **layak**, dengan tiga pembatasan yang harus diterima secara sadar:

19. **Layak untuk pemantauan dan investigasi, belum layak untuk keputusan keselamatan otomatis.** Kualitas posisi (67,1% FLOAT, 1 Hz) cukup untuk melihat dan memutar ulang, tidak cukup untuk menegakkan aturan yang berakibat hukum.
20. **Layak untuk armada dan area, belum layak untuk objek radar.** Perbedaan ini bukan soal teknologi render, melainkan ketersediaan kontrak data dari perangkat.
21. **Layak dengan tim kecil, tetapi tidak layak dengan satu orang.** Kompetensi yang dibutuhkan mencakup web 3D, geospasial, dan backend data — tiga bidang yang jarang dikuasai satu orang secara mendalam.

Dengan tiga pembatasan itu diterima dan dituliskan, sisa pekerjaannya adalah pekerjaan rekayasa biasa: jelas, terukur, dan bisa dipecah menjadi sprint. Itu posisi yang baik untuk memulai — jauh lebih baik daripada proyek yang dijanjikan bisa melakukan segalanya sejak hari pertama.

9 REKOMENDASI DAN LANGKAH LANJUT

9.1 Rekomendasi teknologi

Rekomendasi ini saya susun sebagai keputusan berlapis, bukan satu pilihan tunggal. Alasannya: keputusan yang paling mengikat bukanlah mesin render, melainkan pipeline aset dan skema data — dan keduanya tetap sama apa pun mesin yang dipilih.

Tabel 35. Rekomendasi teknologi per lapis, dengan tingkat keyakinan

Lapis	Rekomendasi	Keyakinan	Alasan pokok
Format aset	**glTF 2.0 (.glb) + Draco + KTX2**	Tinggi	Standar Khronos; format tile wajib 3D Tiles 1.1; ekosistem kompresi matang
Tile geospasial	**3D Tiles 1.1**, di-self-host	Tinggi	Standar OGC; CesiumJS dan 3d-tiles-renderer sama-sama membacanya
Peta raster	**COG (Cloud Optimized GeoTIFF)** + tile XYZ lokal	Tinggi	Bisa dibaca sebagian; tidak perlu unduh 400 MB untuk melihat satu area
Mesin render	**CesiumJS** sebagai basis	Sedang	Skor terbobot tertinggi (3,97); menyelesaikan geodesi & 3D Tiles tanpa kode tambahan
Alternatif mesin	**Three.js + 3d-tiles-renderer**	Sedang	Skor 3,80; lebih fleksibel & talenta lebih mudah; geodesi harus ditulis sendiri
Peta dasar vektor	**MapLibre GL JS** (BSD-3)	Tinggi	Bebas lisensi; Mapbox GL JS sejak v2.0 proprietary
Basis data deret waktu	**PostgreSQL + PostGIS**	Sedang	Sudah umum, satu basis data untuk waktu & geometri; TimescaleDB bila volume naik
Transport real-time	**WebSocket** (MQTT di sisi perangkat)	Tinggi	Cukup untuk ratusan pesan/detik; tidak perlu Kafka
Penyajian	**On-premise, tanpa CDN, tanpa token pihak ketiga**	Tinggi	Risiko R-8; jaringan tambang tidak selalu tersedia

Mengapa CesiumJS, meskipun selisihnya tipis

Selisih 3,97 berbanding 3,80 hanya 4,3% — cukup kecil sehingga keputusannya harus dijelaskan, bukan hanya diumumkan. Ada tiga alasan yang menurut saya membenarkan pilihan CesiumJS untuk **pekerjaan pertama**:

22. Pekerjaan pertama yang bernilai adalah menempatkan tambang dan armada pada koordinat yang benar. Itu justru bagian yang harus ditulis sendiri bila memakai Three.js — dan bagian itu bukan pekerjaan sepele: konversi geodetik, pemilihan origin lokal, penanganan presisi saat kamera jauh, serta terrain.
23. Ortomosaik Minergo berada di UTM 50S dan harus tampil tepat di atas permukaan. CesiumJS menangani konversi dan penempatan ini secara bawaan.
24. Bila kelak dibutuhkan tampilan multi-site atau data elevasi regional, CesiumJS sudah siap; Three.js memerlukan penulisan ulang lapisan bumi.

Namun saya menambahkan satu syarat yang penting: **pipeline aset harus dibangun netral-mesin.** Artinya, konversi ke glTF/3D Tiles dilakukan oleh alat terpisah (glTF-Transform, GDAL, alat tiling), bukan di dalam kode aplikasi. Dengan begitu, bila di tengah jalan CesiumJS ternyata terlalu membatasi tampilan, perpindahan ke Three.js hanya mengganti lapisan render — bukan membangun ulang seluruh sistem.

9.2 Peta jalan bertahap

Urutan fase disusun berdasarkan satu aturan: **setiap fase harus menghasilkan sesuatu yang bisa dinilai pengguna**, bukan hanya kemajuan teknis. Fase yang tidak menghasilkan apa pun yang bisa dilihat berisiko kehilangan dukungan sebelum selesai.

Tabel 36. Peta jalan bertahap dan keluarannya

Tahap	Durasi	Isi	Keluaran yang bisa dinilai
Sprint 0	2 minggu	Audit aset, konversi model, uji beban nyata	Keputusan stack final
Sprint 1	4 minggu	Orthomosaic + RTK live di viewer 3D	Demo UC-1
Sprint 2	3 minggu	Trail, replay 1 jam, panel kendaraan	Demo UC-3 dasar
Sprint 3	3 minggu	Geofence + aturan kecepatan	Demo UC-2
Sprint 4	6 minggu	Radar (bergantung kontrak data vendor)	Demo UC-4 / atau parkir

Perhatikan bahwa Fase 4 (radar) ditempatkan **setelah** tiga fase yang tidak bergantung padanya. Ini keputusan yang disengaja: bila kontrak data radar ternyata tidak diperoleh, tiga fase pertama tetap menghasilkan sistem yang berguna, dan proyek tidak berhenti total. Ini pola yang dalam manajemen risiko disebut mengurangi ketergantungan pada jalur kritis tunggal.

9.3 Estimasi effort

Angka di bawah adalah **asumsi**, bukan komitmen. Dasarnya adalah pengalaman umum proyek sejenis, bukan pengukuran — dan saya menandainya begitu supaya tidak dipakai sebagai dasar kontrak.

Tabel 37. Estimasi effort per fase (jam-orang) — ASUMSI

Fase	Jam-orang	Komposisi tim
Fase 0 — Fondasi data & koordinat	60–90	1 dev + 0,2 surveyor
Fase 1 — Viewer 3D + armada (UC-1)	120–180	1 dev frontend + 1 dev backend
Fase 2 — Geofence & peringatan (UC-2)	80–120	1 dev backend
Fase 3 — Replay historis (UC-3)	100–150	1 dev full-stack
Fase 4 — Integrasi radar / objek terdeteksi (UC-4)	160–320	1 dev + dukungan vendor
Fase 5 — Simulasi (UC-5) & twin permukaan (UC-6)	200–360	2 dev
Total	**720–1220**	—

Total 720–1220 jam-orang. Bila dikerjakan dua orang penuh waktu (sekitar 160 jam-orang per bulan), rentangnya setara 4,5 sampai 7,6 bulan — dengan asumsi tidak ada gangguan prioritas lain. Untuk konteks: tiga fase pertama (fondasi, viewer, geofence) hanya 260–390 jam-orang, yaitu sekitar sepertiga dari total, dan sudah menghasilkan sistem yang bisa dipakai.

CATATAN — Cara membaca estimasi ini

Angka jam-orang di atas **tidak** termasuk: pengadaan perangkat, survei drone tambahan, sertifikasi keamanan, pelatihan pengguna, dan perawatan setelah rilis. Dalam pengalaman umum, keempat hal itu menambah 30–50% dari biaya pengembangan. Angka ini juga mengasumsikan tim sudah menguasai stack web; bila harus belajar dari nol, tambahkan 20–30%.

9.4 Keputusan yang saya minta

Dokumen riset yang berakhir tanpa permintaan keputusan adalah dokumen yang akan didiamkan. Karena itu saya menutup dengan empat keputusan spesifik, masing-masing dengan pemilik yang jelas.

Tabel 38. Keputusan yang diminta, pemilik, dan tenggat yang diusulkan

No	Keputusan	Pemilik	Tenggat usulan	Konsekuensi bila ditunda
K-1	Menyetujui Fase 0–1 (fondasi koordinat + viewer armada) sebagai langkah berikutnya	Kepala R&D + PMO	2 minggu	Tidak ada kemajuan; tim kehilangan momentum dan data tetap tidak terpakai
K-2	Mengirim surat permintaan SDK/object list ke STONKAM dan CISBO	Procurement + R&D	1 minggu	UC-4 tidak bisa dimulai; bila ditunda terlalu lama, Fase 4 harus dikeluarkan dari anggaran tahun ini
K-3	Menetapkan target platform: browser (CesiumJS/Three.js) atau desktop (Unity)	Kepala R&D	3 minggu	Perekrutan dan pelatihan tidak bisa dimulai; risiko R-7 meningkat
K-4	Menyepakati batas penggunaan: pemantauan & investigasi, bukan keputusan hukum otomatis	Manajemen + Safety	3 minggu	Risiko tampilan dipakai melebihi kemampuannya — risiko reputasi dan hukum

9.5 Pertanyaan terbuka untuk vendor dan PIC

Daftar ini adalah keluaran langsung dari gap yang tidak bisa diselesaikan tim R&D sendiri. Setiap pertanyaan disertai alasan mengapa jawabannya menentukan.

Tabel 39. Pertanyaan terbuka dan dampaknya bila tidak terjawab

ID	Pihak	Pertanyaan	Menentukan
V-1	STONKAM	Apakah PSA27 menyediakan object list terstruktur (jarak, sudut, kecepatan) via UART/CAN, dan di mana dokumen protokolnya?	Menentukan apakah UC-4 mungkin sama sekali
V-2	STONKAM	Berapa laju pembaruan objek (Hz) dan berapa jumlah target maksimum yang dilaporkan?	Menentukan apakah objek bisa digambar mulus atau hanya berkedip
V-3	CISBO	Apakah C95M dapat mengeluarkan data target selain status alarm LED? Format dan lajunya?	Menentukan apakah radar CISBO bisa dipakai untuk visualisasi
V-4	CISBO	Klarifikasi jangkauan: halaman resmi menulis 5 m x 6 m, dokumen lama menyebut 120 m. Mana yang benar untuk C95M?	Angka 120 m tidak dapat diverifikasi; salah pakai = salah rancangan
V-5	Quectel / mitra RTK	Apakah LC29H dapat dikonfigurasi ke 5–10 Hz tanpa kehilangan fix rate pada kondisi tambang?	Menaikkan laju sampel mengurangi lompatan 3 m antar posisi
V-6	PIC Survey	Apakah tersedia DSM/point cloud dari survei drone terakhir, atau hanya orthomosaic?	Menentukan apakah twin permukaan (cut/fill) mungkin
V-7	IT / OT	Apakah diizinkan menyajikan viewer 3D on-premise tanpa CDN dan tanpa token pihak ketiga?	Menentukan apakah arsitektur on-premise dapat disetujui

9.6 Penutup

Saya menutup dokumen ini dengan satu pengamatan yang menurut saya paling penting, dan yang mungkin tidak diharapkan dari sebuah studi kelayakan teknologi.

Temuan terbesar dari riset ini bukanlah tentang mesin 3D mana yang lebih baik, atau tentang berapa fps yang bisa dicapai. Temuan terbesarnya adalah bahwa **selisih antara aset yang dirapikan dan tidak dirapikan (8,5 kali) jauh lebih besar daripada selisih antar mesin render (1,5–2 kali)**, dan bahwa **bahan baku digital twin sudah ada di folder Minergo hari ini** — ortomosaik tergeoreferensi, log RTK satu jam perjalanan nyata, model kendaraan berdetail tinggi, dan riwayat telemetri.

Artinya, hambatan utama bukanlah teknologi yang belum dibeli atau perangkat keras yang belum tersedia. Hambatannya adalah pekerjaan yang membosankan dan tidak terlihat: mendefinisikan skema data, merapikan aset, membangun pipeline konversi, dan menulis adapter. Pekerjaan itu tidak menghasilkan demo yang mengesankan, tetapi justru itulah yang menentukan apakah sistem ini akan berfungsi atau hanya akan menjadi presentasi yang bagus.

Satu hal terakhir, tentang apa yang saya sengaja tidak lakukan. Sepanjang dokumen ini ada beberapa tempat di mana saya bisa membuat laporan terlihat lebih lengkap: mengisi panel radar dengan animasi, mengisi kolom DSM dengan perkiraan, atau menuliskan angka fps yang belum diukur. Saya memilih tidak melakukannya, dan menandai setiap tempat itu sebagai celah yang harus ditutup. Bukan karena kerendahan hati, tetapi karena keputusan yang dibangun di atas angka karangan akan gagal di lapangan — dan kegagalan itu akan terjadi setelah anggaran habis, bukan sebelum.

Dokumen ini disusun berdasarkan data yang tersedia di lingkungan R&D Minergo per 8 November 2026, ditambah pengukuran mandiri dan dokumen vendor resmi. Seluruh angka yang ditandai TERVERIFIKASI dapat direproduksi dengan skrip pada Lampiran C. Angka yang ditandai KLAIM VENDOR atau ASUMSI tidak boleh dipakai sebagai dasar komitmen tanpa validasi lanjutan.

LAMPIRAN A — INVENTARIS ASET DAN PENGUKURANNYA

A.1 Model 3D

Setiap berkas dibaca langsung dengan parser glTF (membaca header biner dan JSON chunk), bukan diperkirakan dari nama berkas atau ukuran.

Tabel A-1. Model 3D yang ditemukan beserta properti terukurnya

Berkas	MB	Mesh	Material	Segitiga	Status
HD_3D/HD/ dumptruck_edit.gltf	18.13	14003	3	1.131.975	RUSAK / tidak dapat dimuat apa adanya
Testing_3D/HD/ dumptruck_mapbox .glb	23.59	14003	3	1.131.975	VALID (glTF 2.0 + KHR_draco_mesh_ compression)
Testing_3D/ Excavator/ Excavator.gltf	0.04	26	6	6.452	VALID
CatterPillar_Trcuk_ Diffuse.png	—	—	—	—	Tekstur 3125x3125 PNG belum dikompresi (kandidat KTX2/Basis).
CatterPillar_Trcuk_ Normal.png	—	—	—	—	—

Catatan penting: berkas `dumptruck_edit.gltf` merujuk ke buffer eksternal `dumptruck_edit.bin` sebesar 108 MB yang **tidak ada** di folder tersebut. Model tersebut tidak dapat dimuat apa adanya. Versi yang berfungsi adalah `dumptruck_mapbox.glb` (23,59 MB, sudah terkompresi Draco). Ini gap G-1.

A.2 Data spasial

Tabel A-2. Aset spasial dan parameter georeferensinya

Aset	Parameter	Nilai
Ortomosaik	Dimensi piksel	23.249 × 17.296 px
	Resolusi	1,352 cm/piksel
	Cakupan	314,44 × 233,92 m (7,40 ha)
	Sistem koordinat	EPSG:32750 — WGS 84 / UTM zone 50S
	Sumber georeferensi	chunk eXIf: ModelPixelScale + ModelTiepoint + GeoKeyDirectory
	Tanggal survei	14 Oktober 2026
Titik kontrol	Jumlah titik	22 titik
	Resolusi citra drone / satelit	3,93 cm/px / 14,93 cm/px
	Simpangan maksimum (drone)	0,039 m
	Batas pengukuran	Jarak ke badan jalan pada citra — bukan ke titik GCP terukur langsung
Batas area	Berkas	Minergo.shp (252 byte) + Minergo.prj
	Isi geometri	**0 geometri** — berkas praktis kosong
	Sistem koordinat	WGS 84 geografis

A.3 Data RTK

Tabel A-3. Statistik lengkap log RTK 28 Oktober 2026

Parameter	Nilai
Sumber berkas	rtk-raw-2026-10-28-RTK-Lite.txt + obu-log-20261028-180119-PDS.csv
Jumlah titik	3.601
Durasi	3.602 detik
Jarak tempuh	10.891.3 m
Kecepatan rata-rata	10.82 km/jam
Kecepatan p95	27.49 km/jam
Kecepatan maksimum	35.79 km/jam
Satelit (rata / min / maks)	48.4 / 34 / 53
HDOP (rata / terburuk)	0.4 / 0.56
Umur koreksi RTCM (rata / maks)	1.11 s / 10.0 s
Solusi FIX	1.184 sampel (32.9%)
Solusi FLOAT	2.417 sampel (67.1%)
Jarak antar sampel FIX	3.03 m
Course unik / beku	971 / 1146
Kalimat NMEA (RMC / GGA)	3.604 / 3.601
Checksum salah	4 kalimat
Selisih posisi vs telemetri OBU (rata / maks)	1.27 m / 7.27 m

LAMPIRAN B — SPESIFIKASI PERANGKAT (DARI DOKUMEN RESMI)

B.1 Radar STONKAM PSA27

Sumber: datasheet resmi PDF (S-15). Seluruh angka di bawah adalah **klaim vendor** — saya mengutipnya apa adanya tanpa menguji, karena pengujian perangkat berada di luar lingkup riset ini.

Parameter	Nilai (klaim vendor)
Frekuensi & modulasi	76.00–77.00 GHz, FMCW
Jangkauan deteksi	0,2–40 m (panjang), lebar 0–20 m
Zona deteksi	hingga 5 zona deteksi, dapat dikonfigurasi
Toleransi pengukuran	±15 cm
Sudut pandang	120° horizontal, 14° vertikal
Proteksi lingkungan	IP69K (sensor), IP66 (control box)
Catu daya	10–32 VDC
Ketahanan getaran	ISO 16750-3 2012 (5,9 G)
Rentang suhu	−40…+85 °C (sensor), −20…+70 °C (control box)
Antarmuka	konfigurasi via USB-to-UART atau Wi-Fi 2,4 GHz; trigger input x1; alarm output x1; video AHD/1080P
SDK / protokol data	TIDAK DITEMUKAN

B.2 Radar CISBO C95M

Sumber: halaman produk resmi (S-17). Perhatikan catatan konflik di bagian bawah — ini contoh penting mengapa angka dari dokumen internal harus diverifikasi.

Parameter	Nilai (klaim vendor)
Frekuensi	77 GHz
Jangkauan deteksi	panjang 5 m, lebar 6 m
Sudut pandang	180° horizontal
Proteksi	IP69K
Catu daya	9–36 VDC, ≤400 mA
Antarmuka	LED + buzzer 80–100 dB
SDK / protokol data	TIDAK DITEMUKAN

PERINGATAN — Konflik angka yang tidak boleh diabaikan

Dokumen riset internal sebelumnya menulis 120 m / 100 m dan FOV 150°. Halaman produk resmi saat ini menulis 5 m x 6 m dan H180°. Angka 120 m TIDAK dapat diverifikasi — jangan dipakai. Bila angka 120 m terbawa ke rancangan, jangkauan deteksi akan dilebihkan 20 kali lipat — dan seluruh perhitungan cakupan blind spot menjadi salah. Karena itu angka yang tidak dapat diverifikasi **tidak dipakai** di dokumen ini, sekalipun berasal dari riset internal sendiri.

B.3 Modul GNSS Quectel LC29H

Sumber: datasheet hardware resmi (S-19), berkas lokal 59 halaman.

Parameter	Nilai
Modul	Quectel LC29H (dipakai di RTK_Baru)
Laju pembaruan posisi	1 Hz default, maksimum 10 Hz
Laju data GNSS mentah	hingga 10 Hz
Akurasi pulsa 1PPS	tipikal 100 ns
Antarmuka	UART (NMEA/RTCM/PAIR), I2C
Sumber	Quectel_LC29H_Series_Hardware_Design_V1.0.pdf, hlm. 1–4

Dua hal dari datasheet ini penting untuk rancangan. Pertama, **laju maksimum 10 Hz** berarti masalah lompatan 3 meter di Bab 5.4 sebagian bisa dikurangi hanya dengan mengubah konfigurasi — tanpa perangkat baru. Kedua, **pulsa 1PPS** dapat dipakai untuk menyelaraskan waktu antar perangkat dalam satu kendaraan, yang akan dibutuhkan bila kelak radar dan kamera digabungkan.

B.4 Sensor ultrasonik Howen (sering tertukar)

Parameter	Nilai (klaim vendor)
Teknologi	ULTRASONIK 40 kHz (bukan radar)
Akurasi	1 cm (klaim)
Proteksi	IP67
Antarmuka	kabel sinyal 232
Catatan	Sering tertukar dengan radar. Tidak menghasilkan objek terdeteksi berkoordinat.

Saya mencantumkan perangkat ini justru karena sering **tertukar** dengan radar dalam diskusi internal. Secara teknologi ia berbeda: ultrasonik 40 kHz, bukan gelombang milimeter. Konsekuensinya berbeda pula — jangkauan lebih pendek, dan tetap tidak menghasilkan objek berkoordinat.

B.5 Radar kelas object-list (pembanding)

Untuk menunjukkan bahwa masalah pada Bab 5.6 bukan keterbatasan teknologi radar secara umum, saya mencantumkan satu kelas perangkat yang memang dirancang mengeluarkan object list.

Parameter	Nilai (klaim vendor)
Frekuensi	77/79 GHz (keluarga DRVEGRD)
Antarmuka	Automotive Ethernet + CAN FD
Keluaran	dari raw data sampai object list
Integrasi	ROS, ROS2, Autoware
Status	KLAIM VENDOR (situs resmi) — kontrak datanya tidak dipublikasikan terbuka

Perangkat kelas ini membuktikan bahwa permintaan pada Bab 5.7 bukan permintaan yang mustahil secara teknis — ia hal biasa pada radar otomotif industri. Yang menjadi masalah adalah perangkat spesifik yang dimiliki Minergo saat ini berada di kelas produk yang berbeda: sistem peringatan, bukan sistem persepsi.

LAMPIRAN C — METODOLOGI DAN DATA MENTAH PENGUKURAN RENDER

C.1 Mengapa lampiran ini ada

Angka kinerja di Bab 4 adalah bagian dokumen yang paling mungkin dipertanyakan, dan sepatutnya demikian. Karena itu seluruh metode dan data mentahnya dicantumkan di sini supaya dapat direproduksi atau dibantah. Prinsipnya: angka yang tidak bisa direproduksi tidak layak dipercaya.

C.2 Perangkat dan perangkat lunak

Komponen	Spesifikasi
Mesin	MacBook Apple M1, macOS 26.2
GPU (dibaca dari driver)	ANGLE (Apple, ANGLE Metal Renderer: Apple M1, Unspecified Version)
Ukuran tekstur maksimum	16384
Browser	Google Chrome headless (puppeteer-core 23.11.1)
Resolusi render	1920 x 1080, devicePixelRatio 1
Pustaka	three.js 0.187.0

C.3 Metode pengukuran

Tiga hal dalam metode ini menentukan keabsahan seluruh angka, dan ketiganya lahir dari kegagalan yang saya alami sendiri saat pertama kali mengukur.

25. **Sinkronisasi GPU.** `renderer.render()` hanya menaruh perintah ke antrean; ia kembali sebelum GPU selesai. Tanpa paksaan menunggu, hasilnya mengukur kecepatan CPU mengantre — bukan kecepatan menggambar. Saya memakai `gl.readPixels()` satu piksel setelah setiap render, yang memaksa sinkronisasi. Pengukuran awal saya tanpa ini menghasilkan 27 juta segitiga pada 830 fps, angka yang jelas mustahil dan saya buang.
26. **Pemanasan sebelum pengukuran.** 20 frame pertama dijalankan tanpa dicatat, agar kompilasi shader dan alokasi tekstur tidak masuk hitungan.
27. **Jendela waktu tetap.** Pengukuran berjalan selama 2500 ms dan jumlah frame dihitung, bukan sebaliknya. Ini menghindari bias dari frame pertama yang lebih lambat.

CATATAN METODE — Kesalahan yang saya temukan dan perbaiki sendiri

Pengukuran awal (bench5) menghasilkan angka yang tidak masuk akal. Alih-alih melaporkannya sebagai temuan, saya memeriksa metodenya dan menemukan dua cacat: (a) tidak ada sinkronisasi GPU, dan (b) penghitungan segitiga hanya meng-instance satu kelompok material dari tiga yang ada. Run final (bench7) memperbaiki keduanya dan itulah yang dilaporkan. Saya mencantumkan kegagalan ini karena metode yang menyembunyikan kesalahannya sendiri tidak layak dipercaya.

C.4 Data mentah — seluruh run

Tabel C-1. Seluruh hasil pengukuran, tanpa penyaringan

Skenario	Draw call	Segitiga	Titik	ms/frame	fps
A. GLB as-delivered (14.003 mesh)	14.003	1.131.975	—	70.12	14.3
B. Digabung per material	3	1.131.975	—	8.56	116.8
C. Satu mesh, satu material	1	1.131.975	—	8.25	121.2

Skenario	Draw call	Segitiga	Titik	ms/frame	fps
D1. Instanced x1 (truk HD)	1	1.131.975	—	8.30	120.4
D2. Instanced x24 (truk HD)	1	27.167.400	—	51.04	19.6
D3. Instanced x60 (truk HD)	1	67.918.500	—	124.46	8.0
E. Scene target: 60 truk + 1 jt titik + terrain	3	105.200	1.000.000	11.34	88.2

C.5 Angka turunan dan cara menghitungnya

Besaran	Rumus	Hasil
Biaya per draw call	(70,12 – 8,25) ms ÷ (14.003 – 1)	4.4 mikrodetik
Throughput vertex (60 instance)	67.918.500 segitiga ÷ 124,46 ms	1 juta segitiga/detik
Throughput vertex (24 instance)	27.167.400 segitiga ÷ 51,04 ms	1 juta segitiga/detik
Headroom pada target 30 fps	(33,33 – 11,34) ms ÷ 33,33 ms	66%
Rasio optimasi aset	70,12 ms ÷ 8,25 ms	8,5 kali

Dua angka throughput vertex yang hampir identik (546 dan 546 juta segitiga/detik) bukan kebetulan — itu bukti bahwa pada beban tersebut penyempitnya sudah berpindah dari CPU ke GPU, sehingga penambahan instance berbanding lurus dengan waktu. Ini informasi yang berguna untuk merancang LOD.

C.6 Cara mereproduksi

Seluruh skrip pengukuran disimpan bersama dokumen ini di folder `_work/`. Untuk mereproduksi:

```
# 1. Pasang dependensi
cd _work/bench
npm install three@0.187.0 puppeteer-core@23

# 2. Jalankan (Chrome harus terpasang)
node run_bench7.js

# 3. Hasil
#    bench7_results.json  → data mentah
#    scene_b7.png         → tangkapan layar scene

# Catatan: skrip memakai Chrome headless pada 1920x1080.
# Angka akan berbeda pada perangkat lain — itu wajar dan justru
# alasan pengukuran ulang pada perangkat target diperlukan (Fase 1).
```

Untuk grafik pada Bab 3 dan Bab 4, seluruh angka berasal dari satu berkas `_work/data.py`. Tidak ada angka yang ditulis ulang secara manual di dalam grafik, sehingga tidak mungkin ada ketidakcocokan antara teks, tabel, dan gambar.

LAMPIRAN D — DAFTAR SUMBER

Setiap sumber di bawah ini benar-benar dibuka dan diperiksa isinya, bukan dikutip dari mesin pencari. Untuk sumber berlisensi, yang dibaca adalah berkas lisensi mentah; untuk datasheet, berkas PDF resminya; untuk standar, dokumen spesifikasinya.

Tabel D-1. Daftar sumber dan tingkat verifikasinya

ID	Sumber	Alamat / lokasi	Tingkat
S-01	three.js LICENSE (MIT)	https://raw.githubusercontent.com/mrdoob/three.js/dev/LICENSE	TERVERIFIKASI
S-02	CesiumJS LICENSE.md (Apache-2.0)	https://raw.githubusercontent.com/CesiumGS/cesium/main/LICENSE.md	TERVERIFIKASI
S-03	OGC 3D Tiles 1.1, dok. 22-025r4	https://docs.ogc.org/cs/22-025r4/22-025r4.html	TERVERIFIKASI
S-04	OGC 3D Tiles (halaman standar)	https://www.ogc.org/standards/3dtiles/	TERVERIFIKASI
S-05	Cesium ion — halaman harga	https://cesium.com/platform/cesium-ion/pricing/	TERVERIFIKASI
S-06	Unity — halaman harga	https://unity.com/pricing	TERVERIFIKASI
S-07	Mapbox GL JS LICENSE.txt (Mapbox TOS)	https://raw.githubusercontent.com/mapbox/mapbox-gl-js/main/LICENSE.txt	TERVERIFIKASI
S-08	MapLibre GL JS LICENSE.txt (BSD-3)	https://raw.githubusercontent.com/maplibre/maplibre-gl-js/main/LICENSE.txt	TERVERIFIKASI
S-09	Potree LICENSE (BSD-2)	https://raw.githubusercontent.com/potree/potree/master/LICENSE	TERVERIFIKASI
S-10	Khronos glTF 2.0	https://www.khronos.org/gltf/	TERVERIFIKASI
S-11	KHR_gaussian_splatting (status: Ratified)	https://github.com/KhronosGroup/glTF/blob/main/extensions/2.0/Khronos/KHR_gaussian_splatting/README.md	TERVERIFIKASI
S-12	COPC — Cloud Optimized Point Cloud 1.0	https://copc.io/	TERVERIFIKASI
S-13	EPSG:4978 WGS 84 geosentrik	https://epsg.io/4978	TERVERIFIKASI
S-14	WebGPU — caniuse (86,83% global, Okt 2026)	https://caniuse.com/webgpu	TERVERIFIKASI
S-15	STONKAM PSA27 — datasheet PDF resmi	https://www.stonkam.com/storage/file/20260807/1786081641652919.pdf	KLAIM VENDOR
S-16	STONKAM PSA27 — halaman produk	https://www.stonkam.com/millimeter-wave-radar/7inch-Radar-Detection-System-PSA27	KLAIM VENDOR
S-17	CISBO C95M — halaman produk resmi	https://www.szcisbo.com/truck-bsd-blind-spot-detection-system-with-77ghz-millimeter-wave-radar-c95m	KLAIM VENDOR
S-18	smartmicro — automotive radar (Ethernet/CAN FD, ROS/ROS2)	https://www.smartmicro.com/automotive-radar	KLAIM VENDOR

ID	Sumber	Alamat / lokasi	Tingkat
S-19	Quectel LC29H Hardware Design V1.0 (lokal, 59 hlm)	~/Documents/RTK_Baru/dokumen/Quectel_LC29H_Series_Hardware_Design_V1.0.pdf	TERVERIFIKASI
S-20	Howen HW-H6004A/H6008A datasheet (ultrasonik)	~/Documents/Riset_Jarak/HW-H6004A, HW-H6008A_Datasheet.pdf	KLAIM VENDOR
S-21	Data RTK nyata 28-10-2026 (3.601 titik)	~/Documents/RTK_Baru/apps/rtk-simulator/data.json	TERVERIFIKASI
S-22	Orthomosaic drone + VRT georef	~/Documents/Validasi/map/orthomosaic_georef.vrt	TERVERIFIKASI
S-23	Galat titik RTK vs citra	~/Documents/Validasi/_work/galat_titik.json	TERVERIFIKASI
S-24	Model 3D dump truck (glTF/GLB)	~/Documents/HD_3D/, ~/Documents/Testing_3D/	TERVERIFIKASI
S-25	Pengukuran render three.js 0.187.0 di mesin ini	bench7 (lihat Lampiran C)	TERVERIFIKASI
S-26	3D Gaussian Splatting (Kerbl dkk., 2023)	https://arxiv.org/abs/2308.04079	TERVERIFIKASI
S-27	Dokumen riset internal radar (pembanding)	~/Documents/Riset_Jarak/Riset Radar Deteksi Kendaraan Tambang.docx	ASUMSI — memuat angka yang tidak cocok dengan sumber vendor

D.1 Catatan tentang sumber yang saling bertentangan

Ada satu kelompok sumber yang saling bertentangan dan perlu dijelaskan secara terbuka: dokumen riset radar internal Minergo (S-27) memuat angka jangkauan yang berbeda dari halaman produk resmi vendor untuk perangkat yang sama. Ketika dua sumber bertentangan, saya memakai prinsip: **sumber yang lebih dekat ke pabrikan menang**, dan bila keduanya tidak dapat diverifikasi, angka tersebut tidak dipakai sama sekali.

Konsekuensinya, angka 120 m untuk CISBO C95M — yang muncul di dokumen internal — tidak dipakai di dokumen ini, dan saya jadikan pertanyaan V-4. Ini keputusan yang tidak nyaman karena berarti mengoreksi pekerjaan sendiri, tetapi alternatifnya adalah membangun rancangan di atas angka yang mungkin salah 20 kali lipat.

D.2 Sumber yang dicari tetapi tidak ditemukan

Sama pentingnya dengan daftar yang ditemukan: berikut adalah hal-hal yang sudah dicari dan **tidak** ditemukan. Pencantuman ini mencegah orang lain mengulang pencarian yang sama.

- Dokumen protokol atau SDK untuk object list radar STONKAM PSA27 — tidak ada di situs resmi, tidak ada di halaman unduhan, tidak disebut dalam datasheet.
- Dokumen protokol untuk radar CISBO C95M — halaman produknya hanya menyebut LED dan buzzer sebagai antarmuka.
- DSM atau point cloud dari survei drone Minergo — folder Validasi hanya berisi ortomosaik raster dan data titik kontrol.
- Berkas buffer `dumptruck_edit.bin` (108 MB) yang dirujuk oleh `dumptruck_edit.gltf`.
- Geometri pada `Minergo.shp` — berkas berukuran 252 byte, praktis kosong.
- Skema resmi telemetri OBU — berkas CSV ada, tetapi tanpa dokumentasi field.

LAMPIRAN E — GLOSARIUM DAN SINGKATAN

Glosarium ini disusun untuk pembaca yang tidak berlatar belakang geospasial atau grafika komputer — karena dokumen ini akan dibaca oleh manajemen dan tim operasional, bukan hanya oleh engineer.

3D Tiles — Standar OGC untuk streaming data 3D geospasial secara bertahap sesuai kebutuhan kamera. Versi 1.1 (2023) menjadikan glTF sebagai format tile wajib.

1PPS — One Pulse Per Second — pulsa waktu satu detik sekali dari modul GNSS, dipakai menyelaraskan jam antar perangkat.

COG — Cloud Optimized GeoTIFF — berkas GeoTIFF yang disusun agar bisa dibaca sebagian tanpa mengunduh seluruhnya.

COPC — Cloud Optimized Point Cloud — standar serupa COG, tetapi untuk point cloud.

Draw call — Satu perintah gambar dari CPU ke GPU. Semakin banyak draw call, semakin besar beban CPU per frame — penyebab utama lambat pada Bab 4.

DSM — Digital Surface Model — model permukaan tanah beserta semua objek di atasnya (bangunan, kendaraan, vegetasi). Dibutuhkan untuk twin permukaan.

ECEF — Earth-Centered, Earth-Fixed — sistem koordinat kartesian dengan titik nol di pusat bumi (EPSG:4978). Satuannya meter. Dipakai untuk perhitungan posisi.

ENU — East-North-Up — kerangka lokal dengan sumbu timur, utara, dan atas. Dipakai per tile pada 3D Tiles, dan pada umumnya Y adalah sumbu atas (berbeda dari kerangka kendaraan).

FIX / FLOAT — Status solusi RTK. FIX berarti ambiguitas integer terkunci — akurasi sentimeter. FLOAT berarti belum terkunci — akurasi desimeter sampai meter.

FMCW — Frequency Modulated Continuous Wave — modulasi radar yang mengukur jarak dari pergeseran frekuensi.

Geofence — Batas virtual pada peta. Dipakai untuk menandai zona terlarang, batas kecepatan, atau area operasi.

glTF — GL Transmission Format — standar Khronos untuk model 3D. "JPEG-nya 3D".

GNSS — Global Navigation Satellite System — istilah umum untuk GPS, GLONASS, Galileo, Beidou.

HDOP — Horizontal Dilution of Precision — ukuran geometri satelit. Semakin kecil semakin baik; di bawah 1 sangat baik.

InstancedMesh — Teknik menggambar banyak salinan geometri yang sama dengan satu draw call. Kunci untuk menampilkan puluhan kendaraan.

LOD — Level of Detail — memakai model lebih sederhana saat objek jauh dari kamera.

NMEA 0183 — Format teks standar keluaran penerima GNSS. Kalimat RMC memuat posisi dan kecepatan; GGA memuat kualitas solusi.

Object list — Daftar objek terdeteksi beserta jarak, sudut, dan kecepatannya. Ini yang dibutuhkan visualisasi radar — dan yang tidak dimiliki perangkat saat ini.

Orthomosaic — Citra gabungan hasil foto drone yang sudah dikoreksi geometrinya, sehingga dapat dipakai sebagai peta dengan skala seragam.

RTK — Real-Time Kinematic — teknik GNSS dengan koreksi dari base station, memberikan akurasi sentimeter bila berstatus FIX.

UTM — Universal Transverse Mercator — sistem proyeksi peta dalam meter. Ortomosaik Minergo memakai UTM zona 50S (EPSG:32750).

WebGL2 / WebGPU — Antarmuka grafis browser. WebGL2 tersedia luas; WebGPU lebih baru dan lebih cepat tetapi belum di semua perangkat.

Wireframe — Rancangan tata letak layar tanpa detail visual final — dipakai untuk menguji keputusan informasi sebelum kode ditulis.